

Open-FDD Documentation

Generated by scripts/build_docs_pdf.py

June 3, 2026

- 1 Home
- 2 Open-FDD
 - 2.1 Start here
 - 2.2 AI-assisted bootstrap
 - 2.3 Distribution
 - 2.4 Inspiration
 - 2.5 License
- 3 System Overview
- 4 System overview
 - 4.1 Containers (what runs where)
 - 4.2 Data flow (one building)
 - 4.3 Deploy paths
 - 4.4 PyPI library (secondary)
 - 4.5 Related
- 5 Column map & resolvers
- 6 Column map & resolvers
 - 6.1 1. Plain dict (most IoT / warehouse flows)
 - 6.2 2. Brick TTL (your integration)
 - 6.3 3. Manifest file (ManifestColumnMapResolver)
 - 6.4 4. Composite priority (FirstWinsCompositeResolver)
 - 6.5 Examples in the repo
 - 6.6 See also
- 7 Getting Started
- 8 Getting started
 - 8.1 1. Clone, test, run locally
 - 8.2 2. Docker containers (four apps)
 - 8.3 3. BACnet lab bind (see devices on OT NIC)
 - 8.4 4. AI-assisted deployment — what works / what does not
 - 8.4.1 AI can help with
 - 8.4.2 AI cannot replace
 - 8.4.3 Two different AIs
 - 8.5 5. Operator workflow (after stack is healthy)
 - 8.6 6. Expression / rule references
 - 8.7 PyPI engine only (back of the book)
 - 8.8 Where to read next
- 9 Edge stack layout
- 10 Edge stack layout

- 11 ADR 001 — Core addon serves compiled SPA
- 12 ADR 001 — Core addon serves compiled SPA
 - 12.1 Context
 - 12.2 Decision
 - 12.3 Consequences
 - 12.4 Alternatives considered
 - 12.5 References
- 13 Local Ollama (check-engine)
- 14 Local Ollama (check-engine)
 - 14.1 What local Ollama can access
 - 14.2 What it does not do
 - 14.3 Where Ollama runs
 - 14.4 Checklist (local AI)
 - 14.5 Maintenance
- 15 Docker edge deploy
- 16 Docker edge deploy (Open-FDD)
 - 16.1 Images
 - 16.2 Operational sync (deploy.sh ops)
 - 16.3 Local dev (bensserver)
 - 16.4 Acme / edge deploy
 - 16.4.1 One-time: Caddy on host
 - 16.4.2 Build images + bundle on bensserver
 - 16.4.3 Push to Acme via Ansible
 - 16.4.4 BACnet poll on Acme
 - 16.5 Variables (group_vars / -e)
 - 16.6 Disk maintenance (safe prune)
 - 16.7 What Ansible syncs (Docker path)
 - 16.8 Host services (not Compose app containers)
 - 16.9 Files
 - 16.10 Next steps (registry)
- 17 Engine API
- 18 Engine API
 - 18.1 Rule loading
 - 18.1.1 load_rule
 - 18.1.2 load_rules_from_dir
 - 18.2 RuleRunner
 - 18.2.1 Constructor
 - 18.2.2 add_rule
 - 18.2.3 run
 - 18.3 Bounds helper
 - 18.3.1 bounds_map_from_rule
 - 18.4 Rule types
- 19 Reports API
- 20 Reports API
 - 20.1 Fault report (fault_report)
 - 20.1.1 time_range
 - 20.1.2 flatline_period_range
 - 20.1.3 flatline_period
 - 20.1.4 summarize_fault
 - 20.1.5 summarize_all_faults
 - 20.1.6 analyze_bounds_episodes / analyze_flatline_episodes
 - 20.1.7 build_report_multi_equipment

- 20.1.8 load_rules_for_report /
sensor_cols_from_column_map / print_*
- 20.2 Fault visualization (fault_viz)
 - 20.2.1 get_fault_events
 - 20.2.2 all_fault_events
 - 20.2.3 build_rule_sensor_mapping /
build_sensor_map_for_summarize
 - 20.2.4 plot_fault_analytics / plot_flag_true_bars /
zoom_on_event
 - 20.2.5 run_fault_analytics
- 20.3 Word reports (docx_generator)
 - 20.3.1 events_from_dataframe
 - 20.3.2 events_to_summary_table
 - 20.3.3 build_report
- 20.4 Typical workflow
- 21 API Reference
- 22 API reference
- 23 Column maps (optional ontology keys)
- 24 Column maps and optional ontology keys
 - 24.1 Typical patterns
 - 24.2 See also
- 25 Security
- 26 Security
 - 26.1 Supply chain and dependencies
 - 26.2 Rule YAML and expressions
 - 26.3 Reporting issues
- 27 Ontology labels and column_map
- 28 Ontology labels and column_map
- 29 Context and recordkeeping
- 30 Context and recordkeeping
 - 30.1 Rule
 - 30.2 Good context to commit
 - 30.3 Context anti-patterns
- 31 Operational states
- 32 Operational states
 - 32.1 States
 - 32.2 Why this matters
- 33 Building check-engine light
- 34 Building check-engine light
 - 34.1 Fixed fault codes (letter suffix)
 - 34.2 Equipment families
 - 34.3 How it lights up
 - 34.4 Synergy with expression rules
- 35 Concepts
- 36 Concepts
- 37 How-to Guides
- 38 How-to guides
 - 38.1 Deploy & operate
 - 38.2 Engine (PyPI / offline)
- 39 Desktop app
- 40 Open-FDD Desktop App
 - 40.1 Goal

- 40.2 Install
- 40.3 Launch
 - 40.3.1 Recommended: start-local (repo-local data under stack/local-data)
 - 40.3.2 Restarting start-local and MCP (important)
 - 40.3.3 Built-in AI Agent (/ai-agent)
 - 40.3.4 Manual start (no launcher)
 - 40.3.5 Trusted private LAN (other PCs on the same network)
 - 40.3.6 Gateway HTTP API (Swagger / OpenAPI)
- 40.4 MCP RAG service (agents)
- 40.5 Data ingest quickstart (gateway HTTP API)
 - 40.5.1 1) Create or list sites
 - 40.5.2 2) CSV ingest (file path or upload)
 - 40.5.3 3) Open-Meteo weather ingest
 - 40.5.4 4) Onboard bulk-to-CSV (standalone tool)
 - 40.5.5 5) BACnet ingest (one-shot) and polling
 - 40.5.6 6) Join sources for analysis/plots
- 40.6 CSV timestamp parsing expectations
- 40.7 Data model + BRICK
- 40.8 Feather-first ingestion
- 40.9 Rule loop behavior
- 40.10 Typical desktop workflow (current)
- 40.11 Development
- 41 Agent & operator playbook (bridge + MCP)
- 42 Agent & operator playbook (bridge + MCP)
 - 42.1 Discovery every session should use
 - 42.2 Drivers (BACnet, CSV, weather, ingest)
 - 42.3 Data cleaning & plot-ready metrics
 - 42.4 AI-assisted BRICK data modeling
 - 42.5 FDD (fault detection) and tuning
 - 42.5.1 AI writing the same Python
 - 42.6 MCP action tools (optional writes)
 - 42.7 Built-in Ollama agent (AI Agent tab)
 - 42.8 Query hints for search_docs
- 43 Cloning and porting
- 44 Cloning and porting
 - 44.1 What transfers cleanly
 - 44.2 What changes per deployment
 - 44.3 Recommended first pass on a new machine
 - 44.4 Portability goal
- 45 Verification
- 46 Verification
 - 46.1 1. Unit tests (CI)
 - 46.1.1 Agent shell (optional checkout)
 - 46.1.2 Operator bridge (git checkout)
 - 46.2 2. Operator Rule Lab (Python)
 - 46.3 3. Library YAML rules (optional pip install "open-fdd[engine]")
 - 46.4 4. Small DataFrame smoke test (library)
- 47 PyPI releases (open-fdd)
- 48 PyPI releases (open-fdd)
 - 48.1 openfdd-engine (optional second PyPI name)

- 48.2 Release baseline (PyPI)
- 48.3 1) Before a release of open-fdd
 - 48.3.1 Transition checklist (legacy 0.1.x → 2.x on PyPI)
 - 48.3.2 PyPI upload auth (open-fdd only)
 - 48.3.3 Fallback: API token
 - 48.3.4 If CI shows invalid-publisher or 403
- 48.4 2) Publish open-fdd (commands)
 - 48.4.1 Local dry run
 - 48.4.2 GitHub Actions
- 48.5 3) Publish openfdd-engine (second PyPI project)
- 48.6 Scope policy
- 49 Engine-only deployment and external IoT pipelines
- 50 Engine-only deployment and external IoT pipelines
 - 50.1 Library path — same YAML, any DataFrame
 - 50.1.1 Bring your own column_map or resolver
 - 50.1.2 Manifest file + composite priority
 - 50.2 Standalone playground (optional)
 - 50.3 Summary
- 51 The optional openfdd-engine package
- 52 The optional openfdd-engine package (openfdd_engine)
 - 52.1 Re-exports only
 - 52.2 Mental model
 - 52.3 When to use `pip install open-fdd` vs `openfdd-engine`
 - 52.4 Summary
 - 52.5 See also
- 53 Mode-aware runbooks
- 54 Mode-aware runbooks
 - 54.1 Recommended checks
 - 54.2 Test bench vs production data
- 55 Testing plan
- 56 Testing plan
 - 56.1 Continuous integration
 - 56.2 Local hygiene
 - 56.3 Optional benches
- 57 Operations
- 58 Operations
- 59 Arrow data plane
- 60 Apache Arrow on the edge (what is / is not)
 - 60.1 Pipeline
 - 60.2 Built on Apache Arrow (yes)
 - 60.3 Uses pandas at the boundary (hybrid — not “all Arrow”)
 - 60.4 Not on Arrow (by design — do not overclaim)
 - 60.5 Phase 1 vs 1½ (what landed when)
 - 60.6 Operations
 - 60.6.1 Environment
 - 60.6.2 CLI (host or docker compose exec bridge)
 - 60.6.3 Validation
 - 60.7 Related
- 61 Configuration
- 62 Configuration
 - 62.1 Rule YAML
 - 62.2 Column maps

- 62.3 Environment variables
- 62.4 Related topics
- 63 Contributing
- 64 Contributing to Open-FDD
 - 64.1 Table of Contents
 - 64.2 Code of Conduct
 - 64.3 I Have a Question
 - 64.4 I Want To Contribute
 - 64.4.1 Legal notice
 - 64.5 Reporting Bugs
 - 64.5.1 Before submitting a bug report
 - 64.5.2 How to submit a good bug report
 - 64.6 Suggesting Enhancements
 - 64.6.1 Before submitting
 - 64.6.2 How to submit a good enhancement suggestion
 - 64.7 Contributing Rules (Expression Rule Cookbook)
 - 64.8 Your First Code Contribution
 - 64.9 Improving the Documentation
 - 64.10 Styleguides
 - 64.11 Git workflow (branch → PR → sync)
 - 64.12 Commit Messages
 - 64.13 Join the Project Team
 - 64.14 Attribution
- 65 Expression cookbook (Python / Rule Lab)
- 66 Expression cookbook — Python (Rule Lab)
 - 66.1 Rule shape
 - 66.2 Recipe 1 — Flatline 1 hour
 - 66.3 Recipe 2 — Spread / delta 1 hour
 - 66.4 Recipe 3 — Out of bounds (rolling)
 - 66.5 Recipe 4 — Economizer / mixed air (site-specific)
 - 66.6 Config & bindings
 - 66.7 Shared helpers
 - 66.8 AI drafting tips
- 67 Expression cookbook (YAML / pandas)
- 68 Expression cookbook — YAML / pandas
 - 68.1 Minimal expression rule
 - 68.2 Ontology labels (optional)
 - 68.3 Schedule + weather gates
 - 68.4 Signal scaling (0-1 vs 0-100 %)
 - 68.5 GL36-style example — duct static low at full fan
 - 68.6 Built-in types (non-expression)
 - 68.7 Validation (2.3+)
 - 68.8 Porting YAML → edge Python
- 69 Expression rule cookbooks
- 70 Expression rule cookbooks
 - 70.1 Quick links
- 71 Fault rules (engine / PyPI)
- 72 Fault rules (engine / PyPI)
- 73 Modular Architecture (PyPI)
- 74 Modular architecture (PyPI)
 - 74.1 Modules
 - 74.2 Engine layers

- 74.3 Reports (optional)
- 74.4 See also
- 75 BACnet driver capabilities
- 76 BACnet driver capabilities
 - 76.1 Summary matrix
 - 76.2 Discovery
 - 76.2.1 Who-Is / device scan
 - 76.2.2 Point discovery (one device)
 - 76.2.3 Full discover job
 - 76.3 Read path
 - 76.4 Write path
 - 76.5 Poll driver (historian)
 - 76.6 Point mapping & BRICK model
 - 76.7 Local BACnet server points (edge identity)
 - 76.8 Network / bind checklist
 - 76.9 REST quick reference
 - 76.10 Related
- 77 BACnet
- 78 BACnet
- 79 API reference (index)
- 80 API reference
- 81 Operator Bridge API
- 82 Operator Bridge API
 - 82.1 Health & audit
 - 82.2 Auth
 - 82.3 Rules & FDD
 - 82.4 Rule Lab playground
 - 82.5 BRICK / data model
 - 82.6 BACnet
 - 82.7 Faults (check-engine)
 - 82.8 Timeseries & sites
 - 82.9 Building & host
 - 82.10 Local agent (Ollama)
 - 82.11 Modbus
 - 82.12 PyPI engine API (separate)
 - 82.13 Errors
- 83 Appendix
- 84 Appendix
- 85 Overview
- 86 Fault rules
 - 86.1 Where rules live
 - 86.2 Hot reload
 - 86.3 Rule types
 - 86.4 YAML structure
 - 86.5 Running rules
 - 86.6 Engineering metadata and analytics
 - 86.7 Cookbook
- 87 Test bench rule catalog
- 88 Rule YAML catalog (in this repository)
 - 88.1 open_fdd/tests/fixtures/rules/ (pytest)
 - 88.2 examples/AHU/rules/ (notebooks / demos)
 - 88.3 Related docs

- 89 YAML rules → Pandas (under the hood)
- 90 YAML rules → Pandas DataFrames (under the hood)
 - 90.1 1. YAML never becomes “one giant DataFrame of rules”
 - 90.2 2. From long telemetry rows to a wide DataFrame
 - 90.3 3. What RuleRunner.run does to that DataFrame
 - 90.4 4. How a YAML rule dict becomes pandas expressions
 - 90.5 5. Reloading rules vs DataFrame lifetime
 - 90.6 6. Mental model checklist
- 91 Skills and agent shell
- 92 Skills and agent shell
 - 92.1 Install paths
 - 92.2 Manifest
 - 92.3 Agent shell
 - 92.3.1 Workspace cron
 - 92.3.2 Scheduled wake (mini + critique)
 - 92.3.3 Memory layout
 - 92.4 BACnet toolshed (repo checkout)
 - 92.5 Rule authoring (operator stack)
 - 92.6 Tests (CI)
 - 92.7 Operator dashboard (committed starter)
 - 92.8 Bridge and MCP (generated under workspace/)
- 93 Operator dashboard (Rule Lab)
- 94 Operator dashboard (Rule Lab)
 - 94.1 Layout
 - 94.2 Quick start (recommended — Docker)
 - 94.2.1 Legacy quick start (systemd on host)
 - 94.2.2 UI build modes
 - 94.3 Manual dev (two terminals)
 - 94.4 Production build only
 - 94.5 Tabs (auth may be required)
 - 94.6 Rule Lab modes
 - 94.7 Auth (OT LAN)
 - 94.8 AI maintainers
- 95 Rule Lab — Python storage & shared editing
- 96 Rule Lab — Python storage & shared editing
 - 96.1 Where files live
 - 96.1.1 Git
 - 96.2 Browser (Rule Lab tab)
 - 96.2.1 What you do in the UI
 - 96.2.2 Save flow (existing rule)
 - 96.2.3 Bindings (not on Rule Lab)
 - 96.2.4 Auth
 - 96.3 Scheduled FDD (same Python)
 - 96.4 AI agent — same files
 - 96.4.1 1. AI Agent chat tab (/agent)
 - 96.4.2 2. Agent tools (agent role)
 - 96.4.3 Codex agent shell (repo checkout)
 - 96.5 Data pipeline (BACnet → rules)
 - 96.6 API quick reference
 - 96.7 See also
- 97 Ollama on the edge (hardware)

- 98 Ollama on the edge (hardware)
 - 98.1 Recommended by hardware
 - 98.2 Variables (host_vars / group_vars)
 - 98.3 Acme example
 - 98.4 Maintenance
- 99 Edge Deploy
- 100 Edge deploy (Ansible)
 - 100.1 What is safe on GitHub
 - 100.2 One-time setup
 - 100.3 Deploy commands (examples)
 - 100.4 Inventory hosts
 - 100.5 Acme commissioning layout
 - 100.6 Auth vs SSH
 - 100.7 Agent / Cursor context
- 101 Security Hardening
- 102 Operator Bridge security hardening
 - 102.1 Required for LAN / edge production
 - 102.2 Dev-only (localhost)
 - 102.3 CORS
 - 102.4 BACnet writes
 - 102.5 Rule Lab / diagnostics
 - 102.6 Agent app edit
 - 102.7 Ollama / AI

Generated June 3, 2026

1 Home

2 Open-FDD

Deploy a **building operator stack** on an edge VM or lab host (Acme, bensserver, field Pi): BACnet commission → feather historian → **Python Rule Lab** FDD → dashboard with a **check-engine light**.

The same repo also ships **open-fdd on PyPI** — a pandas YAML rules engine for offline CSV/notebook work. That library is documented in the back of this site under [Fault rules \(engine\)](#).

2.1 Start here

Step	Doc
1	

Step	Doc
	Getting started — deploy checklist, what AI can / cannot do , BACnet NIC bind
2	System overview — containers, data flow, Acme-style topology
3	Docker edge deploy — build images, Ansible deploy.sh, docker, health
4	Local Ollama (check-engine) — on-box AI vs your deployment AI
5	Operator dashboard — Rule Lab, trends, faults
6	BACnet driver capabilities — discover, read, write, poll, mapping
7	Arrow data plane — Feather/Arrow historian: what is and is not columnar
8	Bridge HTTP API — REST reference for integrators

Quick local stack:

```
git clone https://github.com/bbartling/open-fdd.git && cd open-fdd
./scripts/docker_build.sh
./scripts/openfdd_stack.sh up
./scripts/stack_health_check.sh
```

Open `http://<host>/` (Caddy :80 → bridge :8765). Auth: `workspace/auth.env.local` (see [Security hardening](#)).

2.2 AI-assisted bootstrap

Works well with **Cursor**, **Claude Code**, or the repo **agent shell** (`openfdd.toml`, `skills/`, `AGENTS.md`). A deployment assistant can:

- Run tests, build images, and drive Ansible from your laptop
- Discover BACnet (Who-Is, point reads) when the OT NIC is bound correctly
- Import inventory → **BRICK** `model.json`, wire `fdd_input` bindings, draft Rule Lab Python

- Tune thresholds and explain fault episodes from feather + fdd_results.json

It should **not** be treated as a substitute for locked-down production credentials, unchecked BACnet writes on live plant, or sign-off on life-safety interlocks. See the full checklist in [Getting started](#).

2.3 Distribution

Channel	Status
Git + Docker tar	Today — ./scripts/docker_build.sh --save, Ansible load on edge
PyPI open-fdd	Coming soon
GHCR images	Coming soon — Publish Docker addons

2.4 Inspiration

Open-FDD's packaging (supervisor, versioned container images, bind-mounted workspace/ state on a thin host) is **inspired by** the [Home Assistant](#) project and [Home Assistant OS](#). This is an independent BACnet/FDD operator product, not a Home Assistant add-on.

2.5 License

MIT — repository LICENSE.

3 System Overview

4 System overview

Open-FDD on a **git checkout** is an **edge operator product**: four application containers, host Caddy, optional host Ollama, and Ansible deploys like **Acme** and **bensserver**.

4.1 Containers (what runs where)

Image	Port / network	Role
openfdd-bridge	8765 (published)	FastAPI + React SPA: auth, model, Rule Lab API, plots, check- engine, agent routes
openfdd- commission	8767	BACnet commission agent (Who-Is, read/write jobs) — talks to field devices
openfdd- bacnet-poll	network_mode: host	RPM poll driver → samples.csv → feather ingest
openfdd-mcp- rag	8090	Doc/skill retrieval for deployment AI (optional)
ollama/ollama	11434 (optional)	Local LLM for check- engine narrative only — see Local Ollama

On the host (not in app images): **Caddy** :80 → bridge; optional **host Ollama** for GPU; **systemd timers** for FDD batch + feather retention (docker compose exec bridge ...).

State lives under **workspace/** (bind-mounted): data/feather_store/, data/rules_py/, data/model.json, bacnet/, auth.env.local.

4.2 Data flow (one building)

BACnet devices

- commission (discover / points.csv)
- poll (host-network driver)
- ingest → feather wide frames (Apache Arrow IPC – see [Arrow data plane](architecture/arrow_data_plane))
- Rule Lab Python rules (batch / timer)
- fdd_results.json + check-engine (GREEN/YELLOW/RED)
- dashboard + optional local Ollama summary

BRICK model: workspace/data/data_model.ttl synced from BACnet; bridge exposes SPARQL tree/graph APIs. Rules bind logical inputs via rules_store.json + model fdd_input keys.

4.3 Deploy paths

Path	When
Docker + Ansible (recommended)	Acme VM, production edges — Edge deploy (Docker)
Local dev stack	Laptop/server — ./scripts/openfdd_stack.sh up
MSTP lab overlay	compose.bench.yml + workspace/bacnet/commissioning/commission.env NIC bind
Legacy systemd + rsync	Pi without Docker — Ansible deploy.sh all

4.4 PyPI library (secondary)

pip install "open-fdd[engine]" runs **YAML** rules on pandas DataFrames only — no BACnet, no bridge. Use when exporting historian CSVs offline. See [Fault rules \(engine\)](#) and [Expression cookbook \(YAML / pandas\)](#).

4.5 Related

- [Getting started](#)
- [Arrow data plane](#) — historian: built on Arrow vs not
- [Edge stack layout](#)
- [Bridge API](#)

5 Column map & resolvers

6 Column map & resolvers

YAML rules name their **inputs** with **logical keys**—often **Brick** class names (e.g. `Supply_Air_Temperature_Sensor`), but the same **column_map** mechanism supports **Haystack** refs, **DBO** types, **223P** / **s223** strings, or any custom key you put on the input dict. Your **pandas** frame uses **your** column names (`sat`, `AHU1_SupplyTemp`, ...). See [YAML expression cookbook — ontology labels](#).

column_map is the bridge: `dict[str, str]` where each **key** is a **logical label** (Brick class, or a string from the rule's optional **haystack** / **dbo** / **s223** / **223p** fields on that input), and each **value** is the **actual DataFrame column name**. RuleRunner tries those fields in order until one matches a map key; see [YAML expression cookbook](#).

Pass it to `RuleRunner.run(..., column_map=...)`. You can build that dict by hand, load it from a manifest, or **merge** several sources with a composite resolver. Deriving the map from a Brick `.ttl` file with SPARQL is done in **your** services (typically with **rdflib** installed there), not inside the **open-fdd** wheel.

6.1 1. Plain dict (most IoT / warehouse flows)

```
runner.run(
    df,
    column_map={
        "Supply_Air_Temperature_Sensor": "sat",
        "Outside_Air_Temperature_Sensor": "oat",
    },
)
```

No TTL file required if you already know the mapping. Keys must match the **input names** used in your rule YAML (see [Fault rules overview](#)).

6.2 2. Brick TTL (your integration)

If you maintain `data_model.ttl` (or another graph), run SPARQL or graph walks in **your** Python environment (**rdflib**, etc.) and reduce the result to the same `dict[str, str]` you would pass to `RuleRunner.run`.

For **engine-only** notebooks and pipelines, a **manifest** (below) or **plain dict** is usually simpler.

6.3 3. Manifest file (ManifestColumnMapResolver)

JSON or **YAML**: either a flat `logical_name → column_name` object, or a top-level `column_map`: nested mapping.

```
from pathlib import Path
from open_fdd.engine.column_map_resolver import
ManifestColumnMapResolver

r = ManifestColumnMapResolver(Path("my_mapping.yaml"))
runner.run(df, column_map=r.build_column_map(ttl_path=Path(".")))
```

For a raw dict without the resolver class, use `load_column_map_manifest(path)` from the same module.

6.4 4. Composite priority (FirstWinsCompositeResolver)

Run multiple resolvers **in order**; for each logical key, the **first resolver that defines it wins** (for example: a base manifest, then an override file that only adds missing keys). Chain `ManifestColumnMapResolver` instances or add your own `ColumnMapResolver` implementation in application code.

```
from pathlib import Path
from open_fdd.engine.column_map_resolver import (
    FirstWinsCompositeResolver,
    ManifestColumnMapResolver,
)

composite = FirstWinsCompositeResolver(
    ManifestColumnMapResolver("base.yaml"),
    ManifestColumnMapResolver("extras.yaml"),
)
```

```
runner.run(df,  
column_map=composite.build_column_map(ttl_path=Path(".")))
```

Security: There is **no** env-driven “import a resolver class by string” — compose resolvers in code or a small startup script.

6.5 Examples in the repo

Minimal workshop — **examples/column_map_resolver_workshop/** ([folder on GitHub](#)):

Resource	Link
Run	simple_ontology_demo.py
Rule YAML (cookbook-style inputs)	simple_ontology_rule.yaml
Walkthrough	README.md
Top-level index	examples/README.md

AHU / RTU notebooks and sample rules — **examples/AHU/** ([folder on GitHub](#)):

Resource	Link
Sample YAML rules	rules/sensor_bounds.yaml , rules/sensor_flatline.yaml , rules/sat_operating_band.yaml
Notebooks	RTU11_standardized_refactored.ipynb , RTU7_standardized_refactored.ipynb
Sample CSV	RTU11.csv , AHU7.csv
Helpers	openfdd_notebook_helpers_v2.py

More narrative (IoT pipelines, **column_map policy**): [Engine-only & IoT pipelines](#). Curated doc links: [Examples \(repository\)](#).

6.6 See also

- [Getting started](#) — **RuleRunner** + **column_map** in one example
- [Engine-only & IoT pipelines](#) — integration patterns and resolver priority
- [Fault rules overview](#) — YAML **inputs** and Brick-oriented keys
- [Expression rule cookbook](#) — expression rules (same keying model)
- [Examples \(repository\)](#) — notebooks and CSVs under **examples/AHU/**

7 Getting Started

8 Getting started

Use this page as a **deployment checklist** for a building edge (Acme-style VM or lab host). For library-only pandas/YAML work, jump to [Fault rules \(engine\)](#) at the end.

8.1 1. Clone, test, run locally

```
git clone https://github.com/bbartling/open-fdd.git
cd open-fdd
./scripts/build_and_test.sh           # UI + pytest (workspace
bridge tests)
./scripts/docker_build.sh
./scripts/openfdd_stack.sh up         # or: docker compose -f
docker/compose.dev.yml up -d
./scripts/stack_health_check.sh
```

Check	Command / URL
Bridge up	curl -s http://127.0.0.1:8765/health
Stack probe	./scripts/stack_health_check.sh
Dashboard	http://127.0.0.1/ (with Caddy) or :8765
Auth	Copy workspace/auth.env.local.example → auth.env.local

8.2 2. Docker containers (four apps)

Container	You need it when...
bridge	Always — UI, API, Rule Lab, model, faults
commission	BACnet discover/read/write on OT LAN
bacnet-poll	Continuous historian (host network)
mcp-rag	Deployment AI doc search (optional)

Details: [System overview](#). Production push: [Edge deploy \(Docker\)](#).

```
./scripts/docker_build.sh --save
cd infra/ansible && ./deploy.sh docker --limit acme_vm_bbartling
```

```
./deploy.sh ops --limit acme_vm_bbartling # sync TTL, SPARQL
probe, feather check
```

8.3 3. BACnet lab bind (see devices on OT NIC)

Commission and poll use **BACpypes** with a bind address from env.
For bensserver MSTP bench, copy/adjust:

workspace/bacnet/commissioning/commission.env:

```
BACNET_BIND=192.168.204.18/24:47808
# your OT interface IP + UDP 47808
BACNET_INSTANCE=599999
DISCOVER_LOW=5007
DISCOVER_HIGH=5007
DISCOVER_TIMEOUT=30
```

Bring up bench overlay:

```
docker compose -f docker/compose.dev.yml -f docker/
compose.bench.yml up -d
./scripts/setup_local_testbench.sh # discover → model → seed
rules
```

Test discovery from bridge (auth token required in prod): POST /api/bacnet/whois, POST /api/bacnet/discover, then POST /api/bacnet/import-to-model. Wrong BACNET_BIND → zero I-Am responses (fix NIC/IP before blaming rules).

Field Acme uses Ansible commission.env.j2 — do not copy lab commission.env to production.

8.4 4. AI-assisted deployment — what works / what does not

Use **Cursor**, **Claude**, or openfdd-agent-shell with repo skills/ and AGENTS.md. Treat the model as a **commissioning partner**, not an autonomous operator.

8.4.1 AI can help with

Area	Examples
Repo / CI	pytest, build_and_test.sh, Dockerfile fixes, Ansible playbook edits

Area	Examples
BACnet discovery	Who-Is interpretation, points.csv cleanup, bind troubleshooting when OT network is reachable from the edge host
BRICK modeling	Import rows → equipment/points, fdd_input suggestions, SPARQL tree sanity
Rule Lab (Python)	Draft evaluate() rules, flatline/spread/OOB patterns, fault_code from catalog
FDD tuning	Threshold params in rule config, explain spread episodes from feather samples
Deploy	deploy.sh docker, stack_health_check.sh, journal/compose log triage
Docs / skills	Keep skills/*.md aligned with working paths in workspace/

8.4.2 AI cannot replace

Area	Why
Physical OT access	VLANs, BBMD, router MSTP port, firewall 47808 — must be correct on site
Safety sign-off	Writes to live plant, overrides, interlocks — human + RBAC
Secrets	auth.env.local, SSH, API keys — never commit; model must not invent credentials
Guaranteed discovery	If bind/NIC is wrong, no amount of prompting finds devices
Regulatory compliance	Your AHJ / customer MOP for overrides and alarms

8.4.3 Two different AIs

	Deployment AI (Cursor / Claude / MCP)	Local Ollama (on edge)
Purpose	Build, deploy, model, write rules, debug stack	Check-engine light copy + short building insight
Reach		

	Deployment AI (Cursor / Claude / MCP)	Local Ollama (on edge)
	Full repo, Ansible, tests, many bridge tools via MCP	Curated context: faults, trends, catalog codes — Local Ollama
Runs where	Your laptop / CI	Acme GPU host or compose sidecar

8.5 5. Operator workflow (after stack is healthy)

1. **Commission** BACnet → points.csv / inventory APIs.
2. **Import model** → BRICK TTL + model.json; POST /api/model/sync-ttl.
3. **Bind rules** — fdd_input per point; save Python in Rule Lab → rules_py/.
4. **Run batch** — POST /api/rules/batch or host timer; watch **Faults** / check-engine.
5. **Tune** — adjust config thresholds; re-test in Rule Lab playground.

Guides: [Operator dashboard](#), [Rule Lab storage](#), [Skills and agent shell](#).

8.6 6. Expression / rule references

Style	Where
Python (production edge)	Expression cookbook (Python / Rule Lab)
YAML (PyPI engine / CSV)	Expression cookbook (YAML / pandas)

8.7 PyPI engine only (back of the book)

```
pip install "open-fdd[engine]"
pytest open_fdd/tests/engine
```

- [Rules overview](#)
- [Engine API](#)
- [Standalone CSV + pandas](#)

8.8 Where to read next

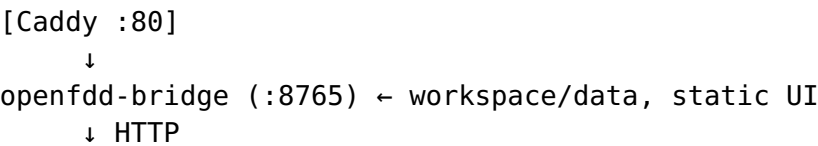
- [System overview](#)
- [Docker edge deploy](#)
- [BACnet driver capabilities](#)
- [Local Ollama](#)
- [Security hardening](#)
- [Bridge HTTP API](#)

9 Edge stack layout

10 Edge stack layout

Three layers on a field host (today: **Ubuntu + Docker CE**; future: thin image under os/).

Layer	Repo path	Role
Host	OS + Caddy + timers	LAN :80, optional GPU Ollama, docker compose exec for FDD loop
Supervisor	supervisor/	Manifest, compose contracts, health — ./ scripts/ openfdd_stack.sh
Apps	docker/ images	bridge, commission, poll, mcp-rag
State	workspace/	Feather, rules, BRICK model, BACnet config (bind-mounted)
Deploy	infra/ansible/	Image tar + workspace sync — deploy.sh docker



openfdd-commission (:8767) ← BACNET_BIND on OT NIC
openfdd-bacnet-poll (host net) → samples → feather
openfdd-mcp-rag (:8090) ← optional doc search

ADR: Core addon serves SPA — UI ships inside openfdd-bridge.

Future os/ Buildroot image: os/README.md (not required for current Acme/bensserver deploys).

11 ADR 001 — Core addon serves compiled SPA

12 ADR 001 — Core addon serves compiled SPA

Status: Accepted

12.1 Context

The operator UI is a Vite/React SPA. We could split a static openfdd-dashboard image or serve the SPA from Caddy on the host.

12.2 Decision

Keep the **compiled React SPA** baked into the **openfdd-bridge image** and served by FastAPI at runtime (one container: UI + API).

12.3 Consequences

Pro	Con
Single image to version and health-check	UI rebuild requires bridge image rebuild
Same auth/CORS/session as API	Larger bridge image
Matches current Acme/bensserver deploy	Separate dashboard image deferred

12.4 Alternatives considered

- **Sidecar static image** — extra compose service; only worth it if UI release cadence diverges from API.
- **Host Caddy file_server** — splits cache headers and deploy paths.

12.5 References

- [Edge stack layout](#)
- `workspace/api/openfdd_bridge/main.py` — SPA fallback routes
- `scripts/build_operator_dashboard.sh`

13 Local Ollama (check-engine)

14 Local Ollama (check-engine)

On the edge, **Ollama is not your commissioning copilot**. It powers the **building check-engine light** narrative: short GREEN/YELLOW/RED explanations and optional insight lines on the home dashboard — using a **fixed fault catalog** the model cannot invent.

Deployment AI (Cursor, Claude, MCP, agent shell on your laptop) owns repo edits, Ansible, BACnet modeling, and Rule Lab drafts. See [Getting started — two AIs](#).

14.1 What local Ollama can access

Source	Used for
GET /api/faults/status	Check-engine color + active codes
GET /api/faults/catalog	Allowed codes only (reject unknown in APIs)
Building insight / zone temps	Cached summaries for dashboard copy
Trend snippets	Recent feather series (bounded context)

Source	Used for
MCP RAG (optional)	Project docs — lighter than full repo checkout

Routes: GET /openfdd-agent/ollama/health, POST /openfdd-agent/chat, GET /openfdd-agent/building-insight. Implementation: agent_routes.py, building_insight.py, fault_catalog.py.

14.2 What it does not do

- Replace Rule Lab execution (Python rules + fdd_runner batch)
- Discover BACnet or edit commission.env bind
- Push Ansible or rebuild Docker images
- Author new fault codes (catalog is fixed: performance, simultaneous heat/cool, sensor, I/O)

Concept: Building check-engine light.

14.3 Where Ollama runs

Setup	Typical host	Bridge env
GPU VM (Acme)	Host systemd Ollama	http://host.docker.internal:11434
Compose sidecar	ollama/ollama service	http://ollama:11434
Disabled	—	Check-engine still works; AI copy falls back

Ansible: deploy.sh ai (host binary) vs openfdd_docker_ollama: true. Variables in infra/ansible/group_vars/.

```
# Dev – Ollama in compose (Linux; avoids host.docker.internal timeouts)
docker compose -f docker/compose.dev.yml -f docker/compose.ollama-smoke.yml --profile ai up -d
```

More hardware notes: Ollama on the edge (deploy).

14.4 Checklist (local AI)



Ollama health (requires auth unless dev bypass):

```
# Production / strict auth – login first, then:
TOKEN=$(curl -s -X POST http://127.0.0.1:8765/api/auth/login \
-H 'Content-Type: application/json' \
-d '{"username":"YOUR_USER","password":"YOUR_PASSWORD"}' | jq -r .access_token)
curl -s http://127.0.0.1:8765/openfdd-agent/ollama/health \
-H "Authorization: Bearer ${TOKEN}"

# Local dev only: unauthenticated curl works when
OFDD_AUTH_DISABLED=1 on trusted localhost
```

- ☐ Model reachable (JSON shows reachable or ok)
 - ☐ Fault catalog page loads; codes are letter suffix (e.g. VAV-C) per fault_catalog.py
 - ☐ Building insight / Agent context mention active fault_code values from FDD, not equipment names
 - ☐ At least one Rule Lab rule has fault_code + enabled binding
 - ☐ POST /api/rules/batch ran after rule edits
 - ☐ Dashboard check-engine matches GET /api/faults/status
 - ☐ Do **not** point production Ollama at the open internet without auth on bridge
-

14.5 Maintenance

```
ollama pull <model> # host
docker compose exec ollama ollama pull <model>
journalctl -u ollama -f # host GPU path
```

Upgrade: bump ollama_version in Ansible group_vars and re-run deploy.sh ai, or docker pull ollama/ollama:<tag>.

15 Docker edge deploy

16 Docker edge deploy (Open-FDD)

Run the bridge, BACnet commission, MCP RAG, and (optionally) BACnet poll as **containers**.

The Open-FDD **app stack does not use systemd units** on Docker edges (openfdd-bridge, openfdd-bacnet-poll, etc. are stopped and replaced by Compose).

Caddy (and optional host **Ollama**) stay on the host for LAN :80 and GPU access. Two small **host timers** run scheduled FDD batch and feather retention via `docker compose exec` — not separate app daemons.

Legacy **pip + rsync + systemd app units** (`./deploy.sh all`) remains for Pi/lab hosts without Docker.

16.1 Images

Image	Role
openfdd-bridge	FastAPI + compiled React SPA
openfdd-commission	BACnet commission HTTP agent
openfdd-bacnet-poll	RPM poll driver (<code>network_mode: host</code>)
openfdd-mcp-rag	Doc search sidecar (:8090)
ollama/ollama	Optional — official image, not built in-tree

State (feather, rules, model, `points.csv`) lives on the host under **workspace/**, bind-mounted into containers.

GHCR publish (later): Images can be pushed to `ghcr.io/bbartling/` via manual GitHub Actions when ready. Edge still uses tar load today. See [Publish Docker addons \(howto\)](#).

16.2 Operational sync (deploy.sh ops)

One-shot playbook for production edges: Docker deploy, safe maintenance prune, workspace ownership fix, **POST /api/model/sync-ttl**, SPARQL tree probe (`query_engine=sparql`), feather size check, bridge log scan, HTTP insurance probes.

```
./scripts/docker_build.sh --save
export SSHPASS='...' # or secrets/acme.env.local
./deploy.sh ops --limit acme_vm_bbartling
```

BRICK reads (/api/model/tree, /graph, /scope) use **rdflib SPARQL** on synced workspace/data/data_model.ttl — not grep/text search on Turtle.

16.3 Local dev (bensserver)

```
cd ~/open-fdd
./scripts/docker_build.sh
./scripts/openfdd_stack.sh up # or: docker compose -f
docker/compose.dev.yml up -d
curl -s http://127.0.0.1:8765/health | jq .

# MSTP test bench (FEC 5007): host-network overlay + commission
poll loop
docker compose -f docker/compose.dev.yml -f docker/
compose.bench.yml up -d
./scripts/setup_local_testbench.sh # discover, model, FDD
rules

# Ollama in compose (recommended on Linux dev –
host.docker.internal often times out)
docker compose -f docker/compose.dev.yml -f docker/compose.ollama-
smoke.yml --profile ai up -d

# Or host Ollama only (works for ./scripts/run_local.sh; bridge-
in-Docker may need the override above)
docker compose -f docker/compose.dev.yml --profile ai up -d
```

Stop: ./scripts/openfdd_stack.sh down OR docker compose -f docker/
compose.dev.yml down

16.4 Acme / edge deploy

16.4.1 One-time: Caddy on host

If the VM has never had Caddy from Ansible:

```
cd infra/ansible
set -a && source secrets/acme.env.local && set +a
./deploy.sh caddy --limit acme_vm_bbartling
```

16.4.2 Build images + bundle on bensserver

```
cd ~/open-fdd
./scripts/build_and_test.sh           # or
build_operator_dashboard.sh prod
./scripts/docker_build.sh --save      # writes docker/dist/openfdd-
images-local.tar.gz
```

16.4.3 Push to Acme via Ansible

```
cd infra/ansible
set -a && source secrets/acme.env.local && set +a
./deploy.sh docker --limit acme_vm_bbartling -e
openfdd_docker_ollama=false
```

Use host systemd Ollama (not the compose ollama service). The bridge container reaches it via host.docker.internal:11434 — ensure sudo systemctl enable --now ollama on the VM.

Optional tag:

```
OPENFDD_IMAGE_TAG=2026.06.01 ./scripts/docker_build.sh --save
./deploy.sh docker --limit acme_vm_bbartling -e
openfdd_docker_image_tag=2026.06.01
```

16.4.4 BACnet poll on Acme

Set in host_vars/acme_vm_bbartling.yml:

```
enable_bacnet_poll_driver: true
```

Re-run ./deploy.sh docker ... — poll container uses **network_mode: host** and reads workspace/bacnet/commissioning/commission.env (OT bind e.g. 10.200.200.185/24:47808).

16.5 Variables (group_vars / -e)

Variable	Default	Purpose
openfdd_docker_image_tag	local	Image tag in compose
openfdd_docker_disable_systemd	true	Stop legacy openfdd-* app units (bridge/mcp/poll/commission)
openfdd_docker_ollama	true	Compose ollama/ollama when

Variable	Default	Purpose
		enable_ollama; use false + ./ deploy.sh ai for host GPU
openfdd_docker_sync_workspace_data	true	Tar-sync workspace/data (rules); set false for image-only
ollama_gpu_mode	cpu	gpu/auto adds NVIDIA device reservation on compose Ollama
openfdd_docker_prune_on_deploy	true	After compose up, run safe image/ container/ network prune
openfdd_docker_prune_unused_images	true	docker image prune -a (only images not used by any container)
openfdd_docker_remove_image_tar	true	Delete docker/ openfdd-images- *.tar.gz on edge after load
openfdd_docker_prune_build_cache	false	Optional docker builder prune

16.6 Disk maintenance (safe prune)

Each ./deploy.sh docker run loads a new image bundle; without cleanup, old openfdd-* layers and the .tar.gz fill small Pi/VM disks.

After the stack is **up**, Ansible runs infra/ansible/scripts/docker_edge_maintenance.sh:

- Prunes **stopped containers, unused networks, dangling images**
- Optionally prunes **all unused images** (docker image prune -a) — safe because running Compose services keep their images
- Optionally removes the loaded **openfdd-images-*.tar.gz** on the edge host

- **Never** runs `docker volume prune` or `docker system prune --volumes` (workspace/feather is bind-mounted, not in named volumes)

Standalone maintenance (no image redeploy):

```
cd infra/ansible
./deploy.sh maintain --limit acme_vm_bbartling
```

Conservative (dangling images only, keep old tags and tar):

```
./deploy.sh maintain --limit acme_vm_bbartling \
-e openfdd_docker_prune_unused_images=false \
-e openfdd_docker_remove_image_tar=false
```

Skip prune during deploy:

```
./deploy.sh docker --limit acme_vm_bbartling -e
openfdd_docker_prune_on_deploy=false
```

16.7 What Ansible syncs (Docker path)

No Python/API git rsync — unlike legacy `./deploy.sh` all (deploy.yml), the docker playbook does **not** push workspace/api, open_fdd, or built UI from source trees.

Artifact	Mechanism	Purpose
App images	docker/dist/openfdd-images-*.tar.gz → docker load	Bridge, commission, poll, MCP
Workspace state	Optional tar of workspace/data (rules, not feather)	Rule store, configs
model.json	Single file copy from control	Site BRICK model
points.csv	From edge_backup/local/...	Commission table
Env / secrets	auth.env.local, templates	Login, BACnet bind, bridge secret
Compose file	Template render	Stack definition

Skip state sync on image-only deploy:

```
./deploy.sh docker --limit acme_vm_bbartling -e
openfdd_docker_sync_workspace_data=false
```

Future: edge docker compose pull from GHCR (see [publish Docker addons](#)) — no tar over SSH.

16.8 Host services (not Compose app containers)

Host unit	Role
caddy	LAN :80 / TLS → 127.0.0.1:8765
openfdd-fdd-loop.timer	Periodic docker compose exec bridge python -m openfdd_bridge.fdd_runner --once
openfdd-feather-retention.timer	Feather prune (host .venv today)
ollama (optional)	GPU inference when openfdd_docker_ollama: false

Logs: docker compose -f ~/open-fdd/docker-compose.yml logs -f
bridge commission — not journalctl -u openfdd-bridge.

16.9 Files

docker/Dockerfile	multi-target build
docker/compose.dev.yml	local bind-mount stack
scripts/docker_build.sh	build + optional tar export
infra/ansible/deploy_docker.yml	
infra/ansible/edge_docker_maintenance.yml	
infra/ansible/tasks/docker_maintenance.yml	
infra/ansible/scripts/docker_edge_maintenance.sh	
infra/ansible/templates/docker-compose.edge.yml.j2	

16.10 Next steps (registry)

Replace docker save / copy / load with a private registry pull when
you have many sites:

```
docker tag openfdd-bridge:local registry.example/openfdd-bridge:
1.2.3
docker push registry.example/openfdd-bridge:1.2.3
```

Ansible then runs docker compose pull instead of loading a tar.

17 Engine API

18 Engine API

Programmatic Python API for loading FDD rules and running the rule engine against pandas DataFrames. Import from `open_fdd.engine.runner` and related modules.

18.1 Rule loading

18.1.1 `load_rule`

Load a single rule from a YAML file.

```
from open_fdd.engine.runner import load_rule

rule = load_rule("path/to/rule.yaml")
# Returns dict: name, type, flag, inputs, params, expression (if
expression type)
```

18.1.2 `load_rules_from_dir`

Load all rules from a directory (all *.yaml files).

```
from open_fdd.engine.runner import load_rules_from_dir

rules = load_rules_from_dir("examples/AHU/rules")
# Returns list of rule dicts
```

18.2 RuleRunner

18.2.1 Constructor

```
from open_fdd.engine.runner import RuleRunner

# From directory
runner = RuleRunner(rules_path="examples/AHU/rules")

# From list of rule dicts
runner = RuleRunner(rules=[rule1, rule2, ...])
```

18.2.2 add_rule

Append a rule to the runner.

```
runner.add_rule(load_rule("new_rule.yaml"))
```

18.2.3 run

Run all rules against a DataFrame. Returns the same DataFrame with added fault flag columns (*_flag).

```
result_df = runner.run(
    df,
    timestamp_col="timestamp",
    rolling_window=6,  # optional global fallback; prefer
params.rolling_window per rule
    params={"units": "metric"},
    skip_missing_columns=False,
    column_map={"oat": "OAT (°F)", "sat": "SAT (°F)"},
)
```

Parameter	Type	Description
df	pandas.DataFrame	Input time-series. Columns = sensor/ command data.
timestamp_col	str	Timestamp column name (default: "timestamp").
rolling_window	int, optional	Global fallback for consecutive samples required to flag; overridden by per- rule params.rolling_window.
params	dict, optional	Override params (e.g. units for bounds rules).
skip_missing_columns	bool	If True, skip rules whose inputs are missing from the DataFrame instead of raising.
column_map	dict, optional	Map from rule input name to DataFrame column name (e.g. from Brick TTL).

Returns: DataFrame with original columns plus fault flag columns (e.g. flatline_flag, bad_sensor_flag). Flag values are 0 or 1.

18.3 Bounds helper

18.3.1 bounds_map_from_rule

Build a bounds map (input → (low, high)) from a bounds rule for a given unit system.

```
from open_fdd.engine.runner import bounds_map_from_rule

rule = load_rule("sensor_bounds.yaml")
bounds = bounds_map_from_rule(rule, units="metric")
# Returns {"Supply_Air_Temperature_Sensor": (4.0, 66.0), ...}
```

18.4 Rule types

Type	Check function	Typical params
bounds	check_bounds	low, high; units (imperial/metric)
flatline	check_flatline	tolerance, window
expression	check_expression	(from rule YAML)
hunting	check_hunting	delta_os_max, ahu_min_oa_dpr, window
oa_fraction	check_oa_fraction	(rule-specific)
erv_efficiency	check_erv_efficiency	(rule-specific)

See [Fault rules overview](#) and the [Expression Rule Cookbook](#) for YAML structure and examples.

19 Reports API

20 Reports API

Programmatic Python API for fault analytics, visualization, and report generation. Used by notebooks and the FDD runner to produce fault summaries, charts, and Word reports. Import from `open_fdd.reports`.

Dependencies: Core reporting uses pandas only. Word (.docx) reports require python-docx (pip install python-docx); without it, `build_report`, `events_from_dataframe`, and `events_to_summary_table` are unavailable.

20.1 Fault report (fault_report)

Fault duration, time ranges, flatline periods, bounds/flatline episode analysis, and multi-equipment report building.

20.1.1 time_range

Return a human-readable time range string for faulted rows.

```
from open_fdd.reports import time_range

s = time_range(df, flag_col="flatline_flag",
timestamp_col="timestamp")
# Returns "2025-01-01 03:00:00 to 2025-01-01 06:15:00" or "-" if
no faults
```

Parameter	Type	Description
df	DataFrame	Has fault flag column and timestamps.
flag_col	str	Name of fault flag column (0/1).
timestamp_col	str	Timestamp column (default: "timestamp").

20.1.2 flatline_period_range

Return (start_ts, end_ts) for the full flatline period (including the window before the first flagged row). Use with `summarize_fault` so fault-period stats match.

```

from open_fdd.reports import flatline_period_range

period = flatline_period_range(df, flag_col="flatline_flag",
timestamp_col="timestamp", window=12)
# Returns (start_ts, end_ts) or None if no faults

```

20.1.3 flatline_period

Same as time_range but for flatline rules: accounts for the rule's window so the returned string covers the actual flat period, not just when the flag is 1.

```

from open_fdd.reports import flatline_period

s = flatline_period(df, flag_col="flatline_flag",
timestamp_col="timestamp", window=12)

```

20.1.4 summarize_fault

Compute fault analytics for a single flag column: total days/hours, hours in fault mode, percent true/false, optional motor runtime, optional sensor stats during fault.

```

from open_fdd.reports import summarize_fault

summary = summarize_fault(
    df,
    flag_col="flatline_flag",
    timestamp_col="timestamp",
    sensor_cols={"SAT": "SAT (°F)", "OAT": "OAT (°F)"},
    motor_col="Supply_Fan_Speed_Command",
    period_range=(start_ts, end_ts), # optional, from
flatline_period_range
)
# Returns dict: total_days, total_hours, hours_*_mode,
percent_true, percent_hours_true,
# hours_motor_runtime (if motor_col), flag_true_* (if
sensor_cols), fault_period_*

```

20.1.5 summarize_all_faults

Run summarize_fault for each flag column and return a dict of summaries. Use with build_report (docx) or custom reporting.

```

from open_fdd.reports import summarize_all_faults

```

```
summaries = summarize_all_faults(df, flag_cols, column_map, ...)
# Returns {flag_col: summary_dict, ...}
```

20.1.6 analyze_bounds_episodes / analyze_flatline_episodes

Analyze contiguous episodes of bounds or flatline faults. Return episode lists with start/end and optional stats.

```
from open_fdd.reports import analyze_bounds_episodes,
analyze_flatline_episodes
```

20.1.7 build_report_multi_equipment

Build a combined report across multiple equipment (e.g. multiple AHUs). Uses rules, column map, and result DataFrames.

```
from open_fdd.reports import build_report_multi_equipment
```

20.1.8 load_rules_for_report / sensor_cols_from_column_map / print_*

Helpers for report workflows: load rules, derive sensor columns from column_map, print column mapping or episode summaries.

20.2 Fault visualization (fault_viz)

Event detection and plotting for notebooks and report charts.

20.2.1 get_fault_events

Return list of (start_iiloc, end_iiloc, flag_name) for contiguous fault regions in a single flag column.

```
from open_fdd.reports import get_fault_events

events = get_fault_events(df, flag_col="flatline_flag")
# Returns [(0, 42, "flatline_flag"), (100, 105,
"flatline_flag"), ...]
```

20.2.2 all_fault_events

Collect events from multiple flag columns, sorted by start index.

```
from open_fdd.reports import all_fault_events

events = all_fault_events(df, flag_cols=["flatline_flag",
"bad_sensor_flag"])
```

20.2.3 build_rule_sensor_mapping / build_sensor_map_for_summarize

Build fault-to-sensor mappings from rules and column_map for use with summarize_fault and plots.

```
from open_fdd.reports import build_rule_sensor_mapping,
build_sensor_map_for_summarize
```

20.2.4 plot_fault_analytics / plot_flag_true_bars / zoom_on_event

Plot fault analytics (e.g. flag-over-time, sensor values during faults). zoom_on_event produces a zoomed plot around a fault event; save the figure path for embedding in Word reports.

```
from open_fdd.reports import plot_fault_analytics,
plot_flag_true_bars, zoom_on_event
```

20.2.5 run_fault_analytics

High-level: run analytics and optionally generate plots for all flags. Wraps the above for notebook-friendly use.

```
from open_fdd.reports import run_fault_analytics
```

20.3 Word reports (docx_generator)

Requires python-docx. Generate Word (.docx) reports with methodology, executive summary, charts, fault summary table, and recommendations.

20.3.1 events_from_dataframe

Convert (start_iiloc, end_iiloc, flag_name) events to a list of dicts with flag, start, end, duration_samples. Use with all_fault_events output.

```
from open_fdd.reports import events_from_dataframe

event_dicts = events_from_dataframe(df, events,
timestamp_col="timestamp")
```

20.3.2 events_to_summary_table

Convert event dicts to a DataFrame suitable for a table (Flag, Start, End, Duration).

```
from open_fdd.reports import events_to_summary_table

table_df = events_to_summary_table(event_dicts)
```

20.3.3 build_report

Build a full Word report: heading, methodology, executive summary, embedded charts, fault summary table, recommendations, optional rules reference.

```
from open_fdd.reports import build_report

report_path = build_report(
    result=result_df,
    events=event_dicts,
    summaries=summaries, # from summarize_all_faults
    output_dir=Path("reports"),
    equipment_name="AHU7",
    plot_paths=["reports/zoom_flatline_1.png"],
    methodology="Optional custom methodology text.",
    executive_summary="Optional pre-written summary.",
    recommendations=["Check sensor calibration.", "Review
setpoints."],
    rules_reference="Optional YAML or text of rules used.",
)
# Returns Path to saved .docx (e.g. reports/
FDD_Report_AHU7_2025-01-15.docx)
```

Parameter	Type	Description
result	DataFrame	RuleRunner output with fault flag columns.
events	list[dict]	From events_from_dataframe(all_fault_events(...)).

Parameter	Type	Description
summaries	dict	From <code>summarize_all_faults</code> .
output_dir	Path	Directory for report and any plots.
equipment_name	str	e.g. "AHU7", "Chiller Plant".
plot_paths	list[str], optional	Paths to plot images to embed.
methodology, executive_summary, recommendations, rules_reference	optional	Override default text.

20.4 Typical workflow

1. Run `RuleRunner` on site/equipment data → result `DataFrame`.
2. `all_fault_events(result, flag_cols)` → events.
3. `events_from_dataframe(result, events)` → event dicts.
4. `summarize_all_faults(result, flag_cols, column_map, ...)` → summaries.
5. Optionally `zoom_on_event` or other plots → save paths.
6. `build_report(result, event_dicts, summaries, output_dir, equipment_name, plot_paths=...)` → .docx.

21 API Reference

22 API reference

The `open-fdd` wheel exposes a **Python API** only (no bundled HTTP server).

Module	Documentation
<code>open_fdd.engine</code>	Engine API — <code>RuleRunner</code> , YAML rules, <code>column_map</code>
<code>open_fdd.reports</code>	Reports API — summaries, plots, optional .docx

`open_fdd.schema` defines fault result/event models used internally by the engine; import from there only if you need typed rows for storage or export.

Extras: `pip install "open-fdd[engine]"` · `pip install "open-fdd[reports]"` · `pip install python-docx` for Word output.

23 Column maps (optional ontology keys)

24 Column maps and optional ontology keys

`open_fdd.engine` does not load Brick TTL or BACnet metadata. You pass `column_map` at `RuleRunner.run` time.

24.1 Typical patterns

1. **Simple dict** — `{"SAT": "RTU_11_DA_T(°F)"}` matches logical names in your YAML inputs.
2. **Manifest YAML** — `ManifestColumnMapResolver / load_column_map_manifest`.
3. **Optional ontology fields** — per-input `brick:`, `haystack:`, `dbo:`, `223p:` in rule YAML; keys in `column_map` must match those strings if you use them.

Example rules under `examples/AHU/rules/` often include `brick:` for readability. That is a **convention**, not a package requirement.

24.2 See also

- [Column map resolvers](#)
- [Expression rule cookbook](#)
- [examples/column_map_resolver_workshop/](#)

25 Security

26 Security

The `open-fdd` PyPI package is a **Python library** for evaluating rules on in-memory (or local) **pandas** data. It does **not** open network ports or ship a hosted service.

26.1 Supply chain and dependencies

- **Pinned versions** — follow your organization’s policy for pinning **open-fdd** and its dependencies (**pandas**, **numpy**, **pyyaml**, **pydantic**) in applications.
 - **Verify PyPI artifacts** — use **twine check** and hashes in lockfiles where applicable.
-

26.2 Rule YAML and expressions

- **Trust model** — only load rule YAML from **trusted** paths (version-controlled configs, sealed containers, or signed bundles). Treat rule files like code.
 - **Expression rules** — use the documented expression surface; avoid piping untrusted strings directly into rule files without review.
-

26.3 Reporting issues

Report security-sensitive bugs through the repository’s **Security policy** (if enabled) or maintainer contact on the project README.

27 Ontology labels and column_map

28 Ontology labels and column_map

The **open-fdd** wheel evaluates rules on **pandas** using **column_map** (dict or manifest). **Brick**, **223P**, **Haystack**, **DBO**, and BACnet-shaped metadata are **not** parsed inside this package — you build the bridge in your pipeline.

Here in this repo: see **Column map resolvers** and the **Expression rule cookbook** for ontology-agnostic rule authoring.

Optional per-input fields **brick**, **haystack**, **dbo**, **s223**, and **223p** are matched against **column_map** in that order (**first match wins**). See [examples/column_map_resolver_workshop/](#) for a minimal end-to-end demo.

29 Context and recordkeeping

30 Context and recordkeeping

Open-FDD documentation should be durable, visible, and reusable across contributors.

30.1 Rule

If context affects repeatable operations, store it in versioned docs instead of leaving it in chat-only memory.

30.2 Good context to commit

- Verification strategy and sweep logic.
- BACnet or naming assumptions that affect **column_map** and diagnostics.
- Cookbook additions and regression examples for expression rules.

30.3 Context anti-patterns

- One-machine-only tribal notes with no repo trace.
- Secrets in Markdown.
- Raw transcript dumps without distillation.

31 Operational states

32 Operational states

Open-FDD operations should explicitly identify environment mode before asserting correctness.

32.1 States

- `TEST_BENCH`: deterministic fake-device conditions or lab-only setup.
- `LIVE_HVAC`: production building telemetry and real-world operating behavior.

32.2 Why this matters

The same observed value can be expected in bench mode and suspicious in live mode. Mode-aware interpretation avoids false positives and noisy operations.

33 Building check-engine light

34 Building check-engine light

The building has one **GREEN / YELLOW / RED** light on the home dashboard. A tree underneath breaks faults down by equipment. **Local Ollama** explains the light using a **fixed** code list — it never invents codes. See [Local Ollama](#).

34.1 Fixed fault codes (letter suffix)

Codes live in `fault_catalog.py` and are served at `GET /api/faults/catalog`. Format: **FAMILY-SUFFIX** where suffix is **1-3 letters** (VAV-C, AHU-B, BLD-D) — **not** numeric codes like VAV-03, which collide with physical equipment names on retrofit sites.

Link graph (fault code → category → cookbook pattern): `GET /api/faults/graph`.

APIs that accept a code reject unknown values. FDD covers four categories only:

Category	Meaning
<code>performance_degradation</code>	Efficiency/capacity drifting from baseline.
<code>simultaneous_heat_cool</code>	Heating and cooling fighting each other.
<code>sensor_fault</code>	Flatline / out-of-range / drift / inconsistent sensors.
<code>io_fault</code>	Command vs feedback mismatch, stuck actuators, stale points.

This is **not** classic BAS alarm-and-dial-out.

34.2 Equipment families

AHU, VAV, HEATPUMP, GEO, CHILLER, DATACENTER, BUILDING — each owns a short code list (browse them on the dashboard **Fault catalog** page).

34.3 How it lights up

Tag a Rule Lab rule with a `fault_code` (e.g. VAV-C for zone sensor flatline) → the scheduled FDD run executes `workspace/data/rules_py/*.py` → results group by family in `GET /api/faults/status`.
ok → green, warning → yellow, critical → red.

34.4 Synergy with expression rules

Each catalog entry lists **cookbook_patterns** (e.g. flatline_1h, spread_1h) that map to templates in [Expression cookbook \(Python\)](#). Rule Lab shows the picker; the Agent context includes `fault_codes` and `fault_code_graph` for Ollama commentary on home **Building insight**.

See also: [Rule Lab — Python storage](#), `skills/building-check-engine/SKILL.md`.

35 Concepts

36 Concepts

Background for using **open-fdd** in pandas-based FDD workflows.

Page	Description
Ontology labels and column_map	Brick, Haystack, DBO, and 223P keys in YAML inputs — bridge to DataFrame columns.
Context and recordkeeping	What belongs in versioned docs vs local notes.
Operational states	Test bench vs live HVAC interpretation.
Building check-engine light	Fixed fault codes + GREEN/YELLOW/RED traffic light the local AI drives.

See also the [Documentation home](#) and [Column map resolvers](#).

37 How-to Guides

38 How-to guides

Deploy and operate the edge stack first; PyPI engine how-tos are listed last.

38.1 Deploy & operate

Guide	Topic
Operator dashboard	Rule Lab, ./scripts/openfdd_stack.sh
Edge deploy (Docker)	Acme / field VMs
Rule Lab storage	rules_py/, bindings, batch
Ollama hardware	GPU/CPU install paths
Skills and agent shell	Cursor/Codex, openfdd.toml, cron
Agent playbook	Bridge + MCP tools
Verification	pytest focus

38.2 Engine (PyPI / offline)

Guide	Topic
PyPI releases	Tags, publishing
Engine-only IoT	RuleRunner on DataFrames
Quick reference	Imports
Cloning and porting	Portable YAML
Danger zone	Expression safety

Historical: [Desktop app \(retired\)](#).

39 Desktop app

40 Open-FDD Desktop App

Retired in 2.4: The monolithic gateway, MCP, and React UI were removed from this repository. New work uses **Skills and agent shell** and generated code under **workspace/**. The content below is historical reference.

40.1 Goal

Run Open-FDD locally with a Python **HTTP gateway** (FastAPI, package `open_fdd.gateway`; colloquially the “bridge”) + MCP + web UI. The built-in AI path is **Codex CLI on the bridge host** via `/ai-agent` and `/openfdd-agent/chat`.

This repository includes a React UI workspace at `apps/desktop-ui` that talks to the gateway on port **8765** by default. The recommended automation path is web-first (gateway + MCP + React UI) on the machine where Open-FDD runs.

The **built-in AI agent** (Codex) is instructed to write **new** code under `workspace/scratch/`; see **Skills and agent shell** for the current layout.

40.2 Install

```
pip install "open-fdd[desktop]"
```

40.3 Launch

40.3.1 Recommended: start-local (repo-local data under stack/local-data)

From the repository root, `scripts/start-local.ps1` (Windows) and `scripts/start-local.sh` (bash) export `OFDD_DESKTOP_DATA_DIR`, `OFDD_MODEL_TTL_PATH`, `OFDD_MODEL_TTL_MIRROR_PATH`, `OFDD_TTL_SYNC_INTERVAL_SECONDS`, and `OFDD_BRIDGE_URL` so `model.json`, Feather chunks, and `data_model.ttl` live under `stack/local-data/` (gitignored) instead of per-user app data.

Both scripts **rebuild** `stack/mcp-rag/index/rag_index.json` from `docs/` before starting `open-fdd-mcp-rag` (roles `all` and `mcp`), so MCP search matches current docs on each launch. Set `OFDD_SKIP_MCP_INDEX_BUILD=1` to skip that step when iterating quickly.

Windows — gateway, MCP RAG, and Vite dev UI each in a new PowerShell window:

```
powershell -ExecutionPolicy Bypass -File .\scripts\start-local.ps1
powershell -ExecutionPolicy Bypass -File .\scripts\start-
local.ps1 -LanHost 192.168.1.10 # LAN dashboard (see Trusted
private LAN)
powershell -ExecutionPolicy Bypass -File .\scripts\start-
```



```
local.ps1 -ListenAll # bind 0.0.0.0 without picking a LAN IP
(set OFDD_BRIDGE_URL or AI Agent Bridge base URL)
```

Single role in the current shell (gateway | mcp | ui | adapter):

```
powershell -ExecutionPolicy Bypass -File .\scripts\start-
local.ps1 -Role gateway
```

Optional parameters: **-BridgeUrl**, **-SyncIntervalSeconds**, **-LanHost <ipv4>** (private LAN dashboard: binds gateway and MCP on **0.0.0.0**, runs Vite with **--host 0.0.0.0**, sets **OFDD_CORS_ALLOW_PRIVATE_LAN=1**, and points **OFDD_BRIDGE_URL**, **OFDD_MCP_REST_BASE**, and **OFDD_UI_PUBLIC_BASE** at **http://<ip>:8765 / : 8090 / :5173**; open inbound **8765**, **8090**, **5173** on the host firewall if other PCs connect), and **-ListenAll** (same bind + Vite **--host 0.0.0.0** and private-LAN CORS default, but leaves public URL env vars unchanged unless you set them yourself).

Bash (macOS / Linux / WSL) — **all** runs gateway + MCP + UI in the background with logs under **stack/local-data/logs/**. The script writes **stack/local-data/openfdd-agent-bootstrap.json** and exports **OFDD_AGENT_BOOTSTRAP_FILE** (same behavior as **start-local.ps1**) so the **AI Agent** tab / **GET /openfdd-agent/context** see **bridge_base**, **mcp_rest_base**, and **ui_public_base**.

```
bash ./scripts/start-local.sh
bash ./scripts/start-local.sh gateway # foreground gateway only
bash ./scripts/start-local.sh --lan-host 192.168.1.10 all
# same LAN defaults as -LanHost on Windows
bash ./scripts/start-local.sh --listen-all all
# bind 0.0.0.0 without --lan-host (set OFDD_BRIDGE_URL or AI Agent
Bridge base URL)
# or: OFDD_LAN_HOST=192.168.1.10 bash ./scripts/start-local.sh
```

Override the bridge URL the same way on all platforms: **export OFDD_BRIDGE_URL=http://127.0.0.1:9999** before running the script (optional).

40.3.2 Restarting start-local and MCP (important)

MCP RAG (open-fdd-mcp-rag, default **127.0.0.1:8090**) is designed like a normal server process: it reads **OFDD_AGENT_BOOTSTRAP_FILE**, **OFDD_MCP_***, and the on-disk **rag_index.json** when it **starts**. It does **not** watch those files for live edits while running—the **start-local** scripts regenerate **rag_index.json** from **docs/** before launching MCP (unless **OFDD_SKIP_MCP_INDEX_BUILD=1**), then **open-fdd-mcp-rag** loads that snapshot until you stop and start it again.

Situation	What to do
	Windows: close the previous gateway, mcp-rag, and

Situation	What to do
You ran start-local again while old windows/PIDs are still up	desktop-ui PowerShell windows (or you risk port conflicts on 8765 / 8090 / 5173 and two different MCP instances). macOS/Linux: stop the old open-fdd-gateway , open-fdd-mcp-rag , and npm run dev jobs (see stack/local-data/logs/*.log for PIDs) before starting new ones.
You rebuilt the doc index (python scripts/build_mcp_rag_index.py ...)	Restart open-fdd-mcp-rag so GET /manifest and search_docs see the new rag_index.json .
You changed bridge URL, MCP port, or bootstrap JSON	Restart gateway and MCP (and usually the UI) so every process agrees on the same env + openfdd-agent-bootstrap.json .

So: **each clean start-local run should mean one trio of processes** (gateway + MCP + UI). Treat “refresh MCP” as **stop the old mcp-rag → start again** (the launcher does the start; you own the stop when re-launching).

Codex on macOS: install with **npm install -g @openai/codex**, then **codex login**. If the bridge cannot find the binary, set **OFDD_CODEX_CMD** to the full path (often under **\$(npm config get prefix)/bin/codex**).

First-time UI: **cd apps/desktop-ui && npm install** once; the launcher runs **npm run dev** for the UI role.

After startup you should have:

- Gateway (same CLI as **open-fdd-desktop-bridge**):
open_fdd.gateway — Swagger **http://127.0.0.1:8765/docs**,
OpenAPI **http://127.0.0.1:8765/openapi.json**
- MCP RAG: **http://127.0.0.1:8090**
- Web UI: Vite default (typically **http://127.0.0.1:5173**); align **OFDD_BRIDGE_URL** / UI env with your bridge if you change ports.

Readiness deep links: **GET /assistant/readiness** builds UI URLs using **OFDD_UI_PUBLIC_BASE** (preferred) or **OFDD_UI_PORT** (defaults to **8080** in code if unset). With **start-local**, set **OFDD_UI_PUBLIC_BASE=http://127.0.0.1:5173** (the Windows script does this) so plot/data-model links match the Vite dev server.

40.3.3 Built-in AI Agent (/ai-agent)

The **AI Agent** tab is the **built-in** Open-FDD AI surface: the bridge runs the **codex CLI** on the host today (same auth model as `codex login / codex login --device-auth`). The UI route **/ai-agent** replaces the older **/openfdd-claw-chat** path (which redirects for bookmarks). Endpoints:

- GET `/local-codex/diagnostics` — `codex login` status, where `/ npm` hints (especially Windows), plus **exec_env** (how `codex exec` is invoked)
- POST `/local-codex/chat` — `codex exec ...` for a simple transcript in the UI

40.3.3.1 Where Codex runs

Layer	What it does
Desktop UI (apps/desktop-ui, Vite / npm)	The browser talks to the bridge over HTTP (<code>/openfdd-agent/chat</code> , <code>/local-codex/*</code> , etc.). The UI does not execute Codex or Python rules locally—it only sends messages and shows replies.
Open-FDD bridge (Python, <code>open_fdd.gateway</code>)	The FastAPI gateway orchestrates each turn: it resolves the <code>workdir</code> , builds <code>stdin/context</code> , and <code>subprocess.runs</code> the codex executable on the same host as the bridge. Code lives in <code>open_fdd/gateway/local_codex_cli.py</code> and <code>open_fdd/gateway/openfdd_agent.py</code> .
codex CLI (OpenAI product)	Usually installed with npm (<code>npm install -g @openai/codex</code>), but at runtime it is a normal OS child process (e.g. <code>codex / codex.cmd</code>), not a long-lived “npm server.” Subscriptions, OAuth/session, model choice, tool execution, and sandbox/approval behavior for that process are owned by Codex ; the bridge supplies argv and env (e.g. <code>OFDD_CODEX_EXEC_APPROVAL</code> , <code>OFDD_CODEX_EXEC_SANDBOX</code> , model overrides) so non-interactive runs can reach localhost and edit the configured repo path.

So: **Python starts Codex; Codex runs the agent turn** under the flags and login you have on that machine. Tightening behavior is done with **Open-FDD env vars** (what we pass into `codex exec`) and/or **Codex’s own config and `codex login`**, not by moving execution into the Vite dev server.

Codex exec sandbox (bridge host): the bridge runs `codex --ask-for-approval ... exec ...` so the agent can call **localhost** (e.g. `http://127.0.0.1:8765`) and edit the `workdir`. Defaults:

OFDD_CODEX_EXEC_APPROVAL=never, **OFDD_CODEX_EXEC_SANDBOX=danger-full-access**. Stricter options: `read-only`, `workspace-write` (with **OFDD_CODEX_WORKSPACE_WRITE_NETWORK=true** the bridge adds `-c sandbox_workspace_write.network_access=true`). Full bypass (only on locked-down automation hosts):
OFDD_CODEX_DANGEROUSLY_BYPASS_APPROVALS_AND_SANDBOX=1. See upstream [Codex sandbox](#).

Built-in agent model routing (POST /openfdd-agent/chat only):

SIMPLE tier uses `--model gpt-5.4-mini` (override **OFDD_CODEX_MODEL_SIMPLE**). **COMPLEX** uses `gpt-5.5` then retries once with `gpt-5.4` if `stderr` looks like an unknown-model error (override **OFDD_CODEX_MODEL_COMPLEX** / **OFDD_CODEX_MODEL_COMPLEX_FALLBACK**). By default, every **successful SIMPLE** turn runs a **second codex exec** using the **COMPLEX primary model** as a final critic/reviewer (disable with **OFDD_AGENT_SIMPLE_COMPLEX_CRITIC=0**; timeout **OFDD_CODEX_EXEC_TIMEOUT_CRITIC**). Optional: **OFDD_CODEX_LLM_CLASSIFY=1** runs another short `codex exec` with the simple model to choose **SIMPLE** vs **COMPLEX** before the main turn (extra latency/cost); **OFDD_CODEX_CLASSIFY_TIMEOUT_S** caps that call (default 120s). The UI **Send ▼** menu can send any message as **human-requested COMPLEX** (strong model regardless of auto-route).

Agent chat thread context: the UI sends the last **120** prior turns plus the new message. The bridge formats them into Codex `stdin`. History size is `min(OFDD_AGENT_CHAT_HISTORY_MAX_TOKENS × 4 chars, OFDD_AGENT_CHAT_HISTORY_MAX_CHARS)` using a rough **~4 characters per token** heuristic (`open_fdd/gateway/local_codex_cli.py`): defaults **OFDD_AGENT_CHAT_HISTORY_MAX_TOKENS=8000** ($\approx 32\text{k}$ UTF-8 bytes for prior turns) and **OFDD_AGENT_CHAT_HISTORY_MAX_CHARS=120000** as a hard ceiling. When over budget, **older turns are dropped** and a short “Earlier messages omitted...” line is prepended. **Rolling summarization** (a **SIMPLE** model compressing old turns + a verbatim tail) is optional product work; Open-FDD does not do it today.

The **AI Agent** tab uses a bridge device-code flow for Codex login status recovery. On completion the bridge writes `$CODEX_HOME/auth.json` (default `~/.codex/auth.json`) for ChatGPT-managed auth so `codex exec` on that host is signed in. OAuth tokens are **not** returned to the browser.

40.3.4 Manual start (no launcher)

If you run **open-fdd-desktop-bridge** / **open-fdd-gateway** without **start-local**, writable storage defaults to the per-user **open-fdd-desktop** directory (see **Data model + BRICK**). Set **OFDD_DESKTOP_DATA_DIR** (and optionally **OFDD_MODEL_TTL_PATH**) yourself if you want a custom root.

```
# terminal 1
open-fdd-desktop-bridge

# terminal 2
open-fdd-mcp-rag

# terminal 3 – after npm install in apps/desktop-ui
cd apps/desktop-ui
npm run dev
```

Production-style static UI (build + static server) is optional; see **apps/desktop-ui** README for **npm run build** and static hosting.

Bridge URL consistency:

- **start-local** sets **OFDD_BRIDGE_URL** for child processes; match the UI's bridge base URL (e.g. **VITE_DESKTOP_BRIDGE_BASE** at build time) to the same host/port.
- In the **AI Agent** tab, **Bridge base URL** (stored in the browser as **ofdd-bridge-base-override**) overrides **VITE_DESKTOP_BRIDGE_BASE** without rebuilding—useful when the UI is opened from another host or over an SSH tunnel.
- The gateway also honors **OFDD_BRIDGE_HOST** / **OFDD_BRIDGE_PORT** if you prefer host/port env vars instead of a full URL.

40.3.5 Trusted private LAN (other PCs on the same network)

The stack defaults to **loopback** (127.0.0.1) so random machines cannot reach your bridge. For a **locked-down office / VLAN** where other workstations should use the UI and API, you intentionally **listen on all interfaces** and point URLs at the **bridge host's LAN IP** (not for the public internet).

One-shot launcher (recommended): powershell -
ExecutionPolicy Bypass -File .\scripts\start-local.ps1 -LanHost 192.168.1.10 or **bash ./scripts/start-local.sh --lan-host 192.168.1.10** all applies the listen addresses, CORS flag, Vite bind, and public URL env vars described below. If you only need **listen on all interfaces** without rewriting URLs, use **-ListenAll** or **bash ./scripts/start-local.sh --listen-all all**, then set **OFDD_BRIDGE_URL** (and related bases) yourself or use the **AI Agent**

→ **Bridge base URL** field in the browser. Ensure **apps/desktop-ui/.env.local** (if present) does not force **VITE_DESKTOP_BRIDGE_BASE** back to localhost when LAN browsers must call the bridge.

Manual steps (equivalent to **-LanHost / --lan-host**):

1. **Gateway listen address** — bind uvicorn to all interfaces, then clients use your LAN IP:
 - export OFDD_BRIDGE_HOST=0.0.0.0
 - export OFDD_BRIDGE_PORT=8765 (optional if default)
 - Browsers and tools call **http://<bridge-lan-ip>:8765** (example **http://192.168.1.10:8765**).
2. **MCP RAG** — same idea if other hosts must call MCP directly (Codex on the bridge often still uses 127.0.0.1):
 - export OFDD_MCP_LISTEN_HOST=0.0.0.0
 - export OFDD_MCP_LISTEN_PORT=8090 (default)
3. **Vite dev server** — so other PCs can load the UI:
 - cd apps/desktop-ui && npm run dev -- --host 0.0.0.0
4. **Where the UI sends API traffic** — set **apps/desktop-ui/.env.local** (or build env) so the browser uses the bridge's LAN address, not localhost:
 - VITE_DESKTOP_BRIDGE_BASE=http://192.168.1.10:8765
 - Or use **AI Agent → Bridge base URL** in the running UI (no rebuild).
5. **Bootstrap / readiness links** — for **GET /openfdd-agent/context**, **OFDD_AGENT_BOOTSTRAP_FILE**, and **OFDD_UI_PUBLIC_BASE**, use URLs that are valid **from the bridge host and from browsers on the LAN** (often **http://<bridge-ip>:8765**, **http://<bridge-ip>:8090**, **http://<ui-host-ip>:5173** or a shared hostname if you use DNS).
6. **CORS** — the bridge only allows localhost origins unless you opt in:
 - **OFDD_CORS_ALLOW_PRIVATE_LAN=1** — allow browser Origin matching common **RFC1918** IPv4 patterns (10/8, 192.168/16, 172.16-31) plus localhost, **or**
 - **OFDD_CORS_EXTRA_ORIGINS=http://192.168.1.20:5173** — comma-separated exact UI origins if you prefer an allowlist.
7. **OS firewall** — open inbound **8765**, **8090**, **5173** (or your ports) on the **private** profile only.

This does **not** add authentication or TLS; treat the bridge like an internal tool on a network you trust.

40.3.6 Gateway HTTP API (Swagger / OpenAPI)

Once the gateway is running locally:

- Swagger UI: **http://127.0.0.1:8765/docs**
- OpenAPI JSON: **http://127.0.0.1:8765/openapi.json**

Use Swagger for endpoint discovery, request body examples, and quick local API testing for Codex-assisted or manual workflows. The Python package lives at `open_fdd/gateway/`; `open_fdd/desktop_bridge/` is a thin compatibility shim for older imports.

40.4 MCP RAG service (agents)

`open-fdd` includes an **MCP-style RAG HTTP service** (REST on **8090** by default): **GET** `/manifest`, **POST** `/tools/search_docs`, **POST** `/tools/search_api_capabilities`, and optional **action** tools that proxy the bridge. Agents and Codex discover it via `mcp_rest_base` in **GET** `/openfdd-agent/context` (from `OFDD_AGENT_BOOTSTRAP_FILE` when you use **start-local**).

Docs for operators and LLM retrieval are indexed under **stack/mcp-rag/**; the **agent operator playbook** (`agent_operator_playbook.md`) is written to match what `search_docs` returns. **Design:** HTTP-first, explicit env, no hot-reload of the JSON index—see **Restarting start-local and MCP** above.

Build the local retrieval index:

```
python scripts/build_mcp_rag_index.py --output stack/mcp-rag/
index/rag_index.json
```

Run MCP RAG locally:

```
open-fdd-mcp-rag
# serves on http://127.0.0.1:8090
```

After changing `rag_index.json`, restart this process (or full **start-local**) so clients see updated chunks.

Key env vars: - `OFDD_MCP_LISTEN_HOST` / `OFDD_MCP_LISTEN_PORT` (defaults `127.0.0.1` / `8090`; bind address for `open-fdd-mcp-rag` and for `/health`'s `mcp_listen_hint`) - `OFDD_MCP_OFDD_API_URL` (default `http://127.0.0.1:8765`) - `OFDD_MCP_OFDD_API_KEY` - `OFDD_MCP_ENABLE_ACTION_TOOLS=true` (required for write/config/ingest proxy tools) - `OFDD_MCP_RAG_INDEX_PATH` (default `./stack/mcp-rag/index/rag_index.json`)

Docker image scaffold: - `stack/Dockerfile.mcp_rag`

Platform drivers: ingest implementations live under `open_fdd/platform/drivers/` (BACnet DIY JSON-RPC, Open-Meteo, CSV). `open_fdd/desktop/drivers/` re-exports the same symbols for backward compatibility.

Headless BACnet loop (cron-friendly): after `pip install -e ".[desktop]"`, run `open-fdd-headless-bacnet` once or loop (see `python -m open_fdd.platform.drivers.headless_bacnet -h`). Modes: local uses IngestService on disk; bridge POSTs to the running gateway at `/ingest/bacnet`.

AI-assisted driver setup: `GET /config/drivers/export` returns a sanitized JSON bundle; `POST /config/drivers/validate` checks a proposed bundle without applying. MCP exposes read tools `drivers_export` and `drivers_validate` that proxy those routes.

40.5 Data ingest quickstart (gateway HTTP API)

Base URL:

- `http://127.0.0.1:8765`

40.5.1 1) Create or list sites

```
curl http://127.0.0.1:8765/sites
curl -X POST http://127.0.0.1:8765/sites -H "Content-Type: application/json" -d '{"name":"HQ"}'
```

40.5.2 2) CSV ingest (file path or upload)

```
# direct path
curl -X POST http://127.0.0.1:8765/ingest/csv \
  -H "Content-Type: application/json" \
  -d '{"site_id":"<site-id>","source":"csv","csv_path":"C:/data/site.csv"}'

# multipart upload
curl -X POST http://127.0.0.1:8765/ingest/csv/upload \
  -F "site_id=<site-id>" \
  -F "source=csv" \
  -F "file=@C:/data/site.csv"
```

40.5.3 3) Open-Meteo weather ingest

```
# set weather config once
curl -X POST http://127.0.0.1:8765/config/weather \
  -H "Content-Type: application/json" \
  -d '{"latitude":42.36,"longitude":-71.06,"timezone":"America/New_York","base_url":"https://archive-api.open-meteo.com/v1/archive"}'

# ingest weather rows
```



```
curl -X POST http://127.0.0.1:8765/ingest/weather \
-H "Content-Type: application/json" \
-d "{\"site_id\":\"<site-id>\",\"days_back\":7}"
```

40.5.4 4) Onboard bulk-to-CSV (standalone tool)

Onboard ingest was moved out of the bridge/UI into a standalone Tkinter tool:

```
python tools/onboard_bulk_download_gui.py
```

Use that GUI to export CSV from Onboard, then import the CSV in Open-FDD via /csv-import (or POST /ingest/csv /upload).

40.5.5 5) BACnet ingest (one-shot) and polling

DIY BACnet server contract (JSON-RPC, model point fields):

POST /ingest/bacnet expects a server implementing client_read_multiple with device_instance + requests[] (object_identifier, property_identifier) and returning values in request order.

If co-running **easy-aso** on the same host for optimization experiments, use its supervisor on **18090** (for example easy-aso-supervisor --port 18090) so Open-FDD MCP RAG can keep default **8090**.

```
# one-shot pull
curl -X POST http://127.0.0.1:8765/ingest/bacnet \
-H "Content-Type: application/json" \
-d "{\"site_id\":\"<site-id>\",\"server_url\":\"http://192.168.204.18:8080\",\"api_key\":\"<token>\"}"

# enable 5-minute poll loop
curl -X POST http://127.0.0.1:8765/config/bacnet \
-H "Content-Type: application/json" \
-d "{\"enabled\":true,\"interval_seconds\":300,\"site_id\":\"<site-id>\",\"server_url\":\"http://192.168.204.18:8080\",\"api_key\":\"<token>\"}"
```

40.5.6 6) Join sources for analysis/plots

```
curl -X POST http://127.0.0.1:8765/timeseries/query \
-H "Content-Type: application/json" \
-d "{\"site_id\":\"<site-id>\",\"sources\":[\"csv\",\"weather\",\"bacnet\"],\"join_on_timestamp\":true,\"join_how\":\"outer\",\"limit\":10000}"
```

40.6 CSV timestamp parsing expectations

Desktop CSV ingest is strict enough to catch bad files but flexible on common field formats:

- Timestamp column auto-detection prefers timestamp, then other time/date-like headers.
- Known timezone abbreviations like EDT, EST, CDT, PDT, etc. are normalized to UTC offsets before parsing.
- Parsing uses UTC-normalized datetimes and drops rows with unparseable timestamps.
- A minimum valid timestamp ratio is enforced; files with mostly bad timestamps return a clear validation error instead of silently importing junk data.

Recommended CSV practice:

- Include a dedicated timestamp column (timestamp preferred).
- Use ISO 8601 when possible (example: 2026-03-18T21:00:00-04:00).
- Avoid mixed timestamp formats inside one file.

If import fails, fix timestamp formatting and retry.

40.7 Data model + BRICK

- **model.json** is the operational on-disk model (CRUD via gateway). Path: **<desktop_data_dir>/model.json**, where **desktop_data_dir** is **OFDD_DESKTOP_DATA_DIR** if set, otherwise a per-user directory **open-fdd-desktop** under the OS app-data root (%APPDATA% on Windows, ~/.local/share on Linux, ~/Library/Application Support on macOS) — see **open_fdd.desktop.storage.paths.desktop_data_dir**.
- Generated BRICK TTL defaults to **<desktop_data_dir>/data_model.ttl**, unless **OFDD_MODEL_TTL_PATH** is set (optional mirror: **OFDD_MODEL_TTL_MIRROR_PATH**; interval: **OFDD_TTL_SYNC_INTERVAL_SECONDS**).
- **scripts/start-local.*** points both model and TTL at **stack/local-data/** in the clone so development data stays with the repo.
- BRICK input mapping for rules resolves through **open_fdd.desktop.services.BrickService**.

40.8 Feather-first ingestion

- CSV, weather, and BACnet drivers write pandas frames into timestamped Feather files under **<desktop_data_dir>/**

- **feather_store** (same **desktop_data_dir** as above — e.g. **stack/local-data/feather_store** when using **start-local**).
- Feather path layout is source/site scoped:
 - `<desktop_data_dir>/feather_store/<safe_source>/<safe_site_id>/<timestamp>_<nonce>.feather`
- Storage remains append/chunk-based (many files per site/source) for reliability and backfill workflows.
- The gateway can still present a site-level joined view for plotting (multi-source virtual merge by timestamp), so operators get a single logical trend frame without rewriting raw files.
- Time-series reads for rules concatenate all Feather files for a selected (source, site_id) pair.
- Point metadata supports external refs like:
 - `feather:///<source>/<site_id>/<metric>`
- These refs are persisted in the model and emitted in TTL via `ofdd:externalReference`.

40.9 Rule loop behavior

- `open_fdd.desktop.rules.run_rule_loop_batched` executes rules with memory-aware chunking.
- If data fits available memory target, full-frame evaluation is used.
- Otherwise, DataFrames are chunked and processed with equivalent rule semantics.
- Chunk size can be user-forced from desktop (`chunk_rows`) or auto-estimated from available memory.
- Batched outputs are concatenated back into one result frame so downstream fault summaries behave the same as single-pass runs.

40.10 Typical desktop workflow (current)

1. Create/select a site in the desktop model tab.
2. Ingest data (CSV import or weather/BACnet fetch) into Feather-backed storage.
3. Confirm points and external references in model + generated BRICK TTL.
4. Run rules from a local YAML rules directory.
5. For large datasets, use batched rule execution (`chunk_rows > 0`, or leave auto-estimation on).
6. Review fault columns and summaries from the merged output frame.

40.11 Development

```
git clone https://github.com/bbartling/open-fdd.git
cd open-fdd
python -m venv .venv
source .venv/bin/activate # Windows: .venv\Scripts\activate
pip install -U pip
pip install -e ".[dev,desktop]"
pytest
```

41 Agent & operator playbook (bridge + MCP)

42 Agent & operator playbook (bridge + MCP)

Use this page as **retrieval fodder** for assistants: it ties **human goals** on the Open-FDD **operator bridge** to **HTTP routes**, **MCP RAG** (POST /tools/search_docs on the MCP REST server), and **execution** (Codex on the bridge host, UI actions). Rebuild the MCP index after editing docs: ./scripts/build_mcp_rag_index.sh, then restart the stack (./scripts/run_local.sh restart --ui-skip).

Defaults (local run_local.sh stack): dashboard <http://127.0.0.1/> when Caddy enabled (else <http://127.0.0.1:8765/>), bridge API <http://127.0.0.1:8765>, MCP RAG <http://127.0.0.1:8090>. Optional Vite HMR: <http://127.0.0.1:5173> only with ./scripts/run_local.sh start --dev (not production parity).

Where Codex writes files: new scripts and helpers go under `workspace/scratch/` (gitignored); durable FDD rules go to `workspace/data/rules_py/` via Rule Lab or rules.save — see [Rule Lab — Python storage](#).

42.1 Discovery every session should use

- GET /health — bridge up.
- GET /assistant/readiness — operator links, plots hints, suggested actions.
- GET /openapi.json or GET /docs — full API surface.

- GET `http://127.0.0.1:8090/manifest` then POST `http://127.0.0.1:8090/tools/search_docs` — indexed how-tos + OpenAPI chunks.
 - GET `/openfdd-agent/context` — merged bootstrap (bridge, MCP, UI, endpoint map) for the built-in Codex agent.
-

42.2 Drivers (BACnet, CSV, weather, ingest)

Human goal: pull field data, map columns, run ingest, debug driver health.

- **BACnet:** see `docs/bacnet/index.md`, gateway driver config and health under bridge routes (search OpenAPI for `bacnet`, `driver`).
 - **CSV / uploads:** POST `/ingest/csv`, POST `/ingest/csv/upload` — body keys per OpenAPI (`site_id`, `source`, `paths` or `multipart`).
 - **Model + TTL:** GET `/model/export`, model CRUD under `/sites`, `equipment`, `points`; TTL sync env `OFDD_MODEL_TTL_PATH`, `OFDD_MODEL_TTL_MIRROR_PATH`.
 - **Execution:** Codex can call `Invoke-RestMethod / curl.exe` against the bridge; long-running ingest may need timeouts. Prefer **preview** responses before destructive writes.
-

42.3 Data cleaning & plot-ready metrics

Human goal: strings with units ("84 °F") break Plotly / FDD; normalize columns.

- **Readiness on plot frames:** responses may include `recommend_clean_metrics` and per-column hints.
 - **Primary API:** POST `/timeseries/clean-metrics` — use **commit: false** first (preview), then **commit: true** after human confirmation.
 - **Plots:** GET `/plots/frame`, GET `/plots/site-frame`, POST `/plots/fdd-frame` — inspect JSON sample + readiness block before tuning rules.
 - **Execution:** agent should sequence preview → show diff summary → commit only if operator agrees.
-

42.4 AI-assisted BRICK data modeling

Human goal: align `model.json` / TTL with BRICK, preserve IDs, keep `external_id` aligned to Feather/CSV headers, set `fdd_input` where rules need it.

- **Authoritative prompt (API):** `open_fdd/assistant/data_model_redesign_prompt.py` — `DATA_MODEL_REDESIGN_SYSTEM_PROMPT`, `import_ready_json` contract.
 - **UI copy:** `workspace/dashboard/src/lib/llm-prompts.ts` — keep in sync for human-facing redesign flows.
 - **Bridge assistant route:** search OpenAPI for data-model and assistant endpoints that return machine JSON output.
 - **SPARQL / BRICK context:** `docs/bacnet-rdf-and-brick.md`, `docs/column_map_resolvers.md`.
 - **Execution:** exports via `GET /model/export`; imports via documented import routes; never invent `site_id` — use `GET /sites` or `readiness`.
-

42.5 FDD (fault detection) and tuning

Human goal: run Python rules, interpret faults, tune thresholds.

- **Rule Lab (/rule-lab):** edit `.py` in the browser; lint/test via `POST /api/playground/lint`, `POST /api/playground/test-rule`, `POST /api/playground/run-script`.
- **On disk:** `workspace/data/rules_store.json` + `workspace/data/rules_py/*.py` — humans and AI share the same files ([Rule Lab storage](#)).
- **Persist + schedule:** `POST /api/rules/save`, `GET /api/rules/saved`, `GET/PUT /api/rules/saved/{id}/source`, `POST /api/rules/batch`; background loop: `python -m openfdd_bridge.fdd_runner --loop` (started by `run_local.sh`).
- **Column mapping:** Data Model tab (`/data-model`) — drag rules onto points; bridge merges `fdd_input` / BRICK keys into `column_map` at run time.
- **Tuning loop:** adjust Python + config → preview in Rule Lab → save → batch run → check building check-engine on `/`.

42.5.1 AI writing the same Python

- **Chat tab (/agent):** `POST /openfdd-agent/chat` — Ollama only; does **not** invoke tools automatically. Use `GET /openfdd-agent/context` for `saved_rules` and tool list.
- **Tool API (agent role):** `POST /openfdd-agent/tool` with `{ "tool": "rules.save", "args": { "name", "code", "fault_code", ... } }` — writes the same JSON + `.py` as Rule Lab Save.
- **Run batch from automation:** `{ "tool": "rules.run_batch" }`.

- **Read source:** GET /api/rules/saved/{id}/source or open workspace/data/rules_py/*.py on the host.
-

42.6 MCP action tools (optional writes)

Read-only RAG is always safe. **Proxy writes** (ingest, config) require MCP server env such as **OFDD_MCP_ENABLE_ACTION_TOOLS** and **OFDD_MCP_OFDD_API_KEY** — see docs/howto/desktop_app.md. Without them, assistants should **only** call the bridge directly with operator keys / localhost trust, not assume MCP mutated data.

42.7 Built-in Ollama agent (AI Agent tab)

- **Chat (browser → bridge):** POST /openfdd-agent/chat — local Ollama on the bridge host; optional history for multi-turn. The dashboard does not call Codex directly.
 - **Context:** GET /openfdd-agent/context — model summary, saved rules, fault codes, MCP hints, tool catalog.
 - **Tools (agent role only):** POST /openfdd-agent/tool — rules.save, rules.run_batch, model CRUD; audited. Chat does not auto-invoke tools.
 - **Repo-local shell (no browser):** openfdd-agent-shell and openfdd-wake run Codex from the checkout with openfdd.toml; see [Skills and agent shell](#).
-

42.8 Query hints for search_docs

Use short natural queries containing **keywords**: BACnet driver, clean-metrics, plots fdd-frame, BRICK import_ready_json, rules run commit, readiness, Feather ingest, operator playbook.

43 Cloning and porting

44 Cloning and porting

The **open-fdd** repository should be portable across workstations and CI images: clone, create a virtual environment, `pip install -e ".[dev]"`, run `pytest`.

44.1 What transfers cleanly

- Automated tests (open_fdd/tests, CI in .github/workflows/ci.yml)
- Example rules under **examples/** and **open_fdd/tests/fixtures/rules/**
- Documentation under **docs/**

44.2 What changes per deployment

- Paths to CSV or Parquet inputs
- **column_map** from your naming convention to DataFrame columns
- Optional Brick TTL or SQL metadata **you** maintain outside this package

44.3 Recommended first pass on a new machine

1. `git clone` and `pip install -e ".[dev]"`
2. `python -c "import open_fdd; print('open_fdd OK')"`
3. `pytest`
4. Run a small example from **examples/README.md**

44.4 Portability goal

Another engineer should be able to reproduce rule behavior from **version-controlled YAML** and a **documented data sample**, without private containers or undisclosed APIs.

45 Verification

46 Verification

Practical checks when authoring or shipping **open-fdd** rules.

46.1 1. Unit tests (CI)

From the repo root with dev dependencies (includes **[engine]** libraries):

```
pip install -e ".[dev]"
pytest open_fdd/tests/engine
```

Expression cookbook regressions live in **open_fdd/tests/engine/test_expression_cookbook.py**.

46.1.1 Agent shell (optional checkout)

When using **packages/openfdd-agent-shell**:

```
pip install -e "packages/openfdd-agent-shell[dev]"
pytest packages/openfdd-agent-shell -q
```

Covers manifest loading, memory truncation, cron schedules, wake lock behavior, and REPL slash commands (hermetic tmp_path workspaces in tests).

46.1.2 Operator bridge (git checkout)

```
./scripts/build_and_test.sh                # vitest + prod UI + pytest
tests/workspace_bridge
./scripts/openfdd_stack.sh up                # Docker stack
(recommended)
# or: ./scripts/run_local.sh restart --ui-skip # legacy systemd
curl -sf http://127.0.0.1/health            # Caddy when enabled
curl -sf http://127.0.0.1:8765/health
```

See [Operator dashboard](#).

46.2 2. Operator Rule Lab (Python)

Storage: workspace/data/rules_store.json + workspace/data/rules_py/*.py — see [Rule Lab storage](#).

- Lint sample rule: POST /api/playground/lint with a minimal evaluate() body.
 - Preview on demo/feather frame: POST /api/playground/test-rule with code, config, optional site_id.
 - Save: POST /api/rules/save (or PUT .../source + POST .../save for updates); confirm .py exists on disk.
 - Batch: POST /api/rules/batch or check workspace/.local-run/fdd_loop.log; confirm GET /api/building/status reflects flagged runs.
 - Agent write (optional): POST /openfdd-agent/tool with { "tool": "rules.save", "args": { ... } } (**agent** role).
-

46.3 3. Library YAML rules (optional pip install "open-fdd[engine]")

For notebooks or IoT pipelines outside the operator stack:

- Load each file with load_rule() or point RuleRunner(rules_path=...) at the directory.
 - Confirm every inputs key has a matching entry in column_map (or manifest) before calling run().
-

46.4 4. Small DataFrame smoke test (library)

```
from pathlib import Path
import pandas as pd
from open_fdd.engine.runner import RuleRunner

df = pd.read_csv("your_sample.csv",
parse_dates=["timestamp"]).set_index("timestamp")
runner = RuleRunner(rules_path=Path("path/to/rules"))
out = runner.run(df, timestamp_col="timestamp",
column_map={"Supply_Air_Temperature_Sensor": "SAT"})
assert any(c.endswith("_flag") for c in out.columns)
```

47 PyPI releases (open-fdd)

48 PyPI releases (open-fdd)

Canonical public install for the 2.x rules engine:

```
pip install open-fdd
```

That distribution ships **open_fdd.engine** (RuleRunner, YAML rules, same stack as the platform FDD loop). **Why does PyPI “Release history” mix 0.1.x and 2.x?** That is still **one** project, **open-fdd**: older **0.1.x** uploads were the legacy, hard-coded pandas era; **2.x** is the current config-driven engine + platform-capable tree. Pip resolves the latest **2.x** unless you pin an old version.

48.1 openfdd-engine (optional second PyPI name)

The repo also contains **packages/openfdd-engine/** (import name **openfdd_engine**): a thin re-export that **depends on open-fdd**. Its **version is kept the same as open-fdd 2.x** (e.g. both 2.0.13) so release numbers stay easy to reason about, even though it is a **separate** PyPI project name.

Publishing uses **.github/workflows/publish-openfdd-engine.yml**: push tag **openfdd-engine-vX.Y.Z** after the **openfdd-engine** PyPI project has a **trusted publisher** for that workflow (same OIDC idea as open-fdd). **workflow_dispatch** builds only; upload runs on matching tags.

48.2 Release baseline (PyPI)

- Check live version:

```
curl -s https://pypi.org/pypi/open-fdd/json | python3 -c "import sys, json; print(json.load(sys.stdin)['info']['version'])"
```

48.3 1) Before a release of open-fdd

1. **Version** — Root **pyproject.toml** → [project] version (e.g. 2.0.11).

2. **openfdd-engine** — Bump `packages/openfdd-engine/pyproject.toml` version to the **same** `X.Y.Z` as `open-fdd`, and set `open-fdd>=X.Y.Z` in dependencies. Tag `openfdd-engine-vX.Y.Z` if you publish that project.

48.3.1 Transition checklist (legacy 0.1.x → 2.x on PyPI)

1. Confirm root `pyproject.toml` sets `open-fdd` to the intended `2.x` version.
2. Merge/push release commits to **master** (or your release branch).
3. Tag `open-fdd-vX.Y.Z` and push the tag.
4. Confirm [open-fdd on PyPI](#) shows `X.Y.Z`.

48.3.2 PyPI upload auth (open-fdd only)

CI uses **PyPI Trusted Publishing** (OIDC from GitHub). Configure **one** trusted publisher on the `open-fdd` PyPI project:

1. [pypi.org](#) → project `open-fdd` → **Manage project** → **Publishing** (not Settings).
2. Add a **GitHub** trusted publisher:
 - **Owner:** `bbartling`
 - **Repository:** `open-fdd`
 - **Workflow name:** `publish-open-fdd.yml`
3. Save.

Workflow file: `.github/workflows/publish-open-fdd.yml`. Tags `open-fdd-v*` trigger upload. `workflow_dispatch` is build/check only (no upload) if configured that way.

Official guide: [Publishing with GitHub Actions OIDC](#).

48.3.3 Fallback: API token

If you cannot use OIDC, pass a project-scoped token, for example:

```
uses: pypa/gh-action-pypi-publish@release/v1
with:
  packages-dir: dist/
  password: ${ secrets.PYPI_OPENFDD_TOKEN }
```

48.3.4 If CI shows invalid-publisher or 403

- Trusted publisher on PyPI doesn't match the **exact** workflow filename or repository.
- Tag points to a commit **before** the OIDC workflow existed — merge fix, retag.

- **Version already on PyPI** — bump version and use a **new** tag.
-

48.4 2) Publish open-fdd (commands)

48.4.1 Local dry run

```
cd /path/to/open-fdd    # repo root
python -m pip install --upgrade pip build twine
python -m build
twine check dist/*
```

48.4.2 GitHub Actions

```
git checkout master    # or your release branch
# bump version in pyproject.toml, commit, push
git tag open-fdd-v2.0.11
git push origin open-fdd-v2.0.11
```

48.5 3) Publish openfdd-engine (second PyPI project)

1. Create project **openfdd-engine** on PyPI (if it does not exist).
 2. Add a **trusted publisher** pointing at workflow **publish-openfdd-engine.yml** (same repository as open-fdd).
 3. Tag **openfdd-engine-vX.Y.Z** (same numbers as **open-fdd-vX.Y.Z**) and push — only after PyPI is configured, or OIDC fails with invalid-publisher.
-

48.6 Scope policy

- **PyPI** ships the **Python packages** (**open-fdd** is the supported public name for 2.x).
- This repository is the **rules engine** only — **YAML rules**, **RuleRunner**, and **column_map** on pandas.

See also: [Engine-only deployment and external IoT pipelines](#).

49 Engine-only deployment and external IoT pipelines

50 Engine-only deployment and external IoT pipelines

Library-only (`pip install "open-fdd[engine]"`): See the **engine documentation** — [Getting started](#), [Expression rule cookbook](#), [Column map & resolvers](#), and this page for pandas integrators.

Integrators who already run **data collection** (historians, MQTT, BAS exports) and their own **modeling** can add **FDD** with `open_fdd.engine.RuleRunner` and the same **YAML** rules you would use in any other context.

Package names: The rules code lives under `open_fdd.engine`. The optional `openfdd-engine` package (`openfdd_engine`) is a thin re-export — not a different engine. See [The optional openfdd-engine package](#).

50.1 Library path — same YAML, any DataFrame

The rule runner is `open_fdd.engine.runner.RuleRunner`. It loads `.yaml` rule files (type: `bounds|flatline|expression|...`, inputs, params, ...).
Authoring references:

- [Expression rule cookbook](#)
- Examples under **examples/** in the repository

Minimal integration pattern

1. Build a **pandas** DataFrame whose columns are your sensor traces (and optional timestamp column).
2. Point **RuleRunner** at a directory of `.yaml` files **or** pass `rules=[...]` dicts.
3. Call `run(df, timestamp_col=..., skip_missing_columns=True, column_map={...})`.
4. Read integer `*_flag` columns (0 / 1) and related outputs per the rule definitions.

column_map — when your naming layer uses Brick-style or vendor tags but DataFrame columns differ (for example temp_sa vs a Brick class label), pass **column_map** from logical key to column name. See `RuleRunner.run` in `open_fdd/engine/runner.py`.

50.1.1 Bring your own column_map or resolver

1. **Plain dict** — `column_map: dict[str, str]` (rule input key → DataFrame column). Pass to `RuleRunner.run(..., column_map=column_map)`.
2. **Brick TTL + SPARQL** — The `open-fdd` wheel does **not** include `rdflib`. If you resolve points from TTL yourself, install `rdflib` in your environment and build a dict before calling `RuleRunner`.
3. **Custom ColumnMapResolver** — Implement `build_column_map(*, ttl_path: Path) -> dict[str, str]` per `open_fdd.engine.column_map_resolver`, or use `ManifestColumnMapResolver` / `FirstWinsCompositeResolver` for manifest-driven maps.

Priority / policy: The engine uses **one** `column_map` per run. Resolve ambiguity in your pipeline before calling `RuleRunner`.

50.1.2 Manifest file + composite priority

- `load_column_map_manifest(path)` — reads `.json` or `.yaml`. Accepts a flat object or a nested `column_map`: mapping.
- `ManifestColumnMapResolver(path)` — exposes the manifest via the resolver protocol.
- `FirstWinsCompositeResolver(r1, r2, ...)` — first resolver to supply a key wins.

Examples: `examples/column_map_resolver_workshop/` — `simple_ontology_demo.py`.

Install

```
pip install "open-fdd[engine]"
# or from a checkout (dev extras include PyYAML and pydantic for
rules):
pip install -e ".[dev]"
```

50.2 Standalone playground (optional)

- `examples/engine_iot_playground/` — README, rules, sample CSV, `run_demo.py`, notebook.
-

50.3 Summary

Approach	You bring	Open-FDD provides
RuleRunner in Python	DataFrame + YAML rules	Fault columns and structured outputs
Batch / notebook	CSV or query results	Same semantics as any other caller

For package layout, see [Modular architecture](#).

51 The optional openfdd-engine package

52 The optional openfdd-engine package (openfdd_engine)

Open-FDD has **one rules implementation**: YAML files evaluated on **pandas** via **RuleRunner** in **open_fdd.engine**. The **packages/openfdd-engine/** tree (PyPI name **openfdd-engine**, import **openfdd_engine**) is a **thin re-export** of selected symbols from **open_fdd.engine** — useful when procurement or legacy imports expect the **openfdd_engine** package name.

52.1 Re-exports only

openfdd_engine does not duplicate engine logic. Its **__init__.py** imports from **open_fdd.engine** and lists them in **__all__**. New public APIs belong in **open_fdd.engine** first; the shim gains them through re-export.

- **Versioning**: **open-fdd** on PyPI is authoritative; **openfdd-engine** pins **open-fdd** `>=...` when published.

52.2 Mental model

Layer	What it is	Typical import
Core engine		from open_fdd.engine.runner import RuleRunner

Layer	What it is	Typical import
	Loads YAML rules, runs checks on a DataFrame	
Optional shim	Same classes, different package name	from openfdd_engine import RuleRunner

52.3 When to use `pip install open-fdd` vs `openfdd-engine`

Default:

```
pip install open-fdd
```

```
from open_fdd.engine.runner import RuleRunner, load_rule,
load_rules_from_dir
from open_fdd.engine.column_map_resolver import
load_column_map_manifest
```

Optional `openfdd-engine` — only if you need the separate PyPI project or editable shim:

```
cd packages/openfdd-engine
pip install -e .
```

See [PyPI releases \(open-fdd\)](#).

52.4 Summary

Goal	Install	Import
Normal use	<code>pip install open-fdd</code>	<code>open_fdd.engine.*</code>
Optional second package	published <code>openfdd-engine</code> or editable shim	<code>openfdd_engine.*</code>

52.5 See also

- [Engine-only deployment and external IoT pipelines](#)
- [Modular architecture](#)
- [Expression rule cookbook](#)
- `packages/openfdd-engine/README.md`

53 Mode-aware runbooks

54 Mode-aware runbooks

This repository is the **open_fdd rules engine** only. There is no `--mode collector|model|engine|full` Docker orchestration in-tree.

54.1 Recommended checks

Goal	Command
Verify install	<code>python -c "import open_fdd; print('ok')"</code>
Run unit tests	<code>pytest</code>
Editable dev install	<code>pip install -e ".[dev]"</code>

54.2 Test bench vs production data

Treat **synthetic** or **lab** CSVs as a different **operational state** from **production** BMS exports: tighten tolerances, window sizes, and alert routing in production. See [Operational states](#).

55 Testing plan

56 Testing plan

Engine-focused testing for the **open-fdd** package.

56.1 Continuous integration

On every PR and push to **main**, **master**, or **develop**:

1. Install `pip install -e ".[dev]"`
2. Build combined docs text (`python scripts/build_docs_pdf.py --no-pdf`) for consistency checks
3. Run `pytest` on `open_fdd/tests`

4. Dry-run `python -m build + twine check` for `open-fdd` and `packages/openfdd-engine`

56.2 Local hygiene

- Add **pytest** cases for new rule types, edge cases in expressions, and column-map behavior.
 - Keep **fixtures** small and deterministic under `open_fdd/tests/fixtures/rules/`.
-

56.3 Optional benches

If you maintain a separate hardware or BACnet lab, keep its automation **outside** this repository and pass normalized **DataFrames** into **RuleRunner** here.

57 Operations

58 Operations

Engine-focused notes for tests and local runbooks.

Page	Description
Testing plan	What CI runs; local pytest expectations.
Mode-aware runbooks	Rules engine vs custom orchestration in your own jobs.

59 Arrow data plane

60 Apache Arrow on the edge (what is / is not)

Open-FDD's **edge historian** uses **Feather files** (Apache Arrow IPC on disk) inside the **openfdd-bridge** Docker image. The thin host OS (os/) does not ship Arrow; neither does the **PyPI** open-fdd wheel (pandas + YAML only).

This page is the honest map for operators and integrators: what is columnar/Arrow-native today (Phases **1** and **1½**), and what is intentionally **not**.

60.1 Pipeline

flowchart LR

```
poll[BACnet poll CSV] --> ingest[bacnet_poll_ingest]
ingest --> shard[Feather shards]
shard --> compact[compact / compact_all]
compact --> latest[latest.feather]
latest --> batch[FDD batch one read per site]
batch --> rules[Rule Lab per-row evaluate]
```

60.2 Built on Apache Arrow (yes)

Component	How Arrow is used
On-disk format	latest.feather / shard-*.feather = Arrow IPC via pyarrow (pyarrow>=17 in bridge image)
Shard merge	read_site_table() → pyarrow.concat_tables (threaded read optional)
Compaction	Merge shards → single Feather; compact_all() can run per site in parallel
Retention prune	Prefer pyarrow.compute timestamp filter before rewrite (pandas fallback)
Wide-frame reads	read_site(..., columns=[...]) column-prunes at Feather read

Component	How Arrow is used
FDD batch I/O	One Feather read per site per batch; column union from rule bindings

Implementation: [workspace/api/openfdd_bridge/feather_store.py](#).

60.3 Uses pandas at the boundary (hybrid — not “all Arrow”)

Step	Why pandas
Rule Lab / plots	pd.DataFrame for operator APIs and evaluate(row, ...)
Dedupe / sort	Timestamp dedupe after concat (single materialization)
Demo CSV fallback	load_frame_for_run when no feather yet
PyPI engine	Offline YAML rules — separate from bridge storage

Arrow reduces **I/O and merge** cost; pandas remains the **operator and rule** surface.

60.4 Not on Arrow (by design — do not overclaim)

Area	Reality
Rule Lab default	Row-by-row Python evaluate() — not multi-core vectorized unless the rule author vectorizes inside the script
“All CPU cores”	True mainly for parallel site compact/prune and optional OFDD_ARROW_IO_THREADS on concat/read — not for arbitrary commissioning Python
Vectorized Rule Lab mode	Not shipped — future optional mode beside row-Python
Polars historian	Not shipped — would replace dataframe APIs across bridge
Parquet-only store	Not shipped — Feather shards + latest.feather stay the edge layout
GPU Arrow	Not used

Area	Reality
Host OS image	No Arrow dependency on bare-metal OS; only in containers

Defensible marketing: Arrow-backed Feather historian with parallel site maintenance and efficient batch loads; Rule Lab stays row-Python for commissioning clarity.

Overclaim to avoid: “FDD uses all cores” or “everything runs in Arrow compute.”

60.5 Phase 1 vs 1½ (what landed when)

Phase	Scope
1	Feather = Arrow IPC; pyarrow in bridge image; pickle fallback only if pyarrow missing (ARM edge note in requirements-edge-armv7.txt)
1½	Arrow concat + compute prune; deferred ingest compact; compact_all + workers; column-pruned read_site; FDD one load per site ; docs + scripts/ bench_feather_compact.sh

60.6 Operations

60.6.1 Environment

Variable	Default	Purpose
OFDD_ARROW_IO_THREADS	0	pyarrow.set_cpu_count for Feather read/concat
OFDD_FEATHER_COMPACT_WORKERS	1	Parallel site compact / prune
OFDD_FEATHER_COMPACT_ON_INGEST	0	1 = compact every poll (legacy hot-path)
OFDD_FEATHER_COMPACT_SHARD_THRESHOLD	8	Auto-compact when loose shards $\geq N$
OFDD_FEATHER_MAX_GIB	0	Disk cap (enforce_max_bytes)

Variable	Default	Purpose
OFDD_FEATHER_RETENTION_DAYS	90	Row retention (prune)

60.6.2 CLI (host or docker compose exec bridge)

```
python -m openfdd_bridge.feather_store --compact
python -m openfdd_bridge.feather_store --maintain
```

Important: Ingest usually **defers** compaction. Schedule `--compact` or `--maintain` in cron or `scripts/docker_maintenance.sh` so shards do not sprawl.

60.6.3 Validation

```
./scripts/openfdd_edge_validate.sh --quick # health, SPARQL,
Ollama hello, pytest, logs
./scripts/bench_feather_compact.sh # local timing smoke
```

60.7 Related

- [BACnet capabilities](#) — poll → ingest → feather
- [System overview](#) — container data flow
- [Local Ollama](#) — check-engine AI (separate from Arrow)

61 Configuration

62 Configuration

This page describes how to **configure YAML rules** and engine behavior when using the `open_fdd` library directly. There is **no** required platform database or HTTP service in this repository.

62.1 Rule YAML

Rules are loaded with `load_rule()` / `RuleRunner` (see [Rules overview](#)).

Typical fields include:

Field	Role
<code>name, description</code>	Human-readable metadata
<code>equipment_type</code>	Optional filter for which equipment a rule applies to
<code>type</code>	Check type (bounds, flatline, expression, ...)
<code>params</code>	Check-specific parameters (thresholds, rolling_window, ...)
<code>column_map</code>	Mapping from logical point names to DataFrame columns (dict or manifest path)

Use `fdd_strict_rules`-style tightening in **your** caller if you want stricter validation of column maps and dtypes before evaluation (see [Expression rule cookbook](#)).

62.2 Column maps

- **Inline dict** — simplest: { "SAT": "SupplyAirTemp" } keys match placeholders in the rule.
 - **Manifest YAML** — see [Column map resolvers](#) for `ManifestColumnMapResolver` and composite patterns.
-

62.3 Environment variables

The `open-fdd` wheel does **not** require a fixed `OFDD_*` namespace for `RuleRunner`. Your **application** may use env vars to locate rule directories or data files; that is entirely under your control.

62.4 Related topics

- [Rules overview](#)
- [Engine-only / IoT](#)
- [Getting started](#)

63 Contributing

64 Contributing to Open-FDD

First off, thanks for taking the time to contribute!

Open-FDD is in **Alpha**. The most valuable contributions right now are **bug reports** and **FDD rules**—especially from mechanical engineers and building professionals who can add and refine fault-detection rules using the [Expression Rule Cookbook](#). All types of contributions are encouraged.

Phase focus: Alpha emphasizes **rule correctness**, **engine APIs**, and **documentation** for integrators embedding `open_fdd` in their own data pipelines. Beta (planned) will broaden cookbook coverage and optional modeling guides. See the [Table of Contents](#) for different ways to help. Please read the relevant section before contributing; it helps maintainers and keeps things smooth for everyone.

If you like the project but don't have time to contribute, that's fine. Other ways to support it: - Star the [repository](#) - Share it with colleagues or at meetups - Refer to Open-FDD in your project's readme or documentation

64.1 Table of Contents

- [Code of Conduct](#)
 - [I Have a Question](#)
 - [I Want To Contribute](#)
 - [Reporting Bugs](#)
 - [Suggesting Enhancements](#)
 - [Contributing Rules \(Expression Rule Cookbook\)](#)
 - [Your First Code Contribution](#)
 - [Improving the Documentation](#)
 - [Styleguides](#)
 - [Git workflow \(branch → PR → sync\)](#)
 - [Commit Messages](#)
 - [Join the Project Team](#)
-

64.2 Code of Conduct

This project expects everyone to be respectful and constructive. By participating, you agree to uphold a positive environment. Please report unacceptable behavior by opening an issue or contacting the repository maintainers.

64.3 I Have a Question

Before asking, read **[this site's documentation](#)** (pip install open-fdd), and search [issues](#).

If you still need help:

- Open an [issue](#).
- Provide as much context as you can (what you're trying to do, what you ran, what happened).
- Include relevant versions: Python, OS, and **open-fdd** release or git commit if known.

We'll respond as soon as we can.

64.4 I Want To Contribute

64.4.1 Legal notice

When contributing, you agree that you have authored 100% of the content, have the necessary rights to it, and that your contribution may be provided under the project license (MIT).

64.5 Reporting Bugs

Bug testing is a top priority in Alpha. Good bug reports help us fix issues quickly.

64.5.1 Before submitting a bug report

- Use the latest version (main branch or latest release).
- Confirm the bug is in **open-fdd** (the rules engine) and not in your environment (e.g. wrong Python version, bad `column_map`). Check the [documentation](#) and [I Have a Question](#) first.
- Search [issues](#) to see if the bug is already reported.

- Collect:
 - **Stack trace** (Traceback) if applicable
 - **OS and platform** (e.g. Linux, macOS, Windows; x86, ARM)
 - **Versions**: Python and how you ran the engine (pip install, editable install, etc.)
 - **Steps to reproduce** and, if possible, sample input/config
 - Whether you can reproduce it every time and on an older version

64.5.2 How to submit a good bug report

- **Security issues**: Do **not** report security vulnerabilities in public issues. Email the repository owner or open a private security advisory on GitHub.
- Open a new issue. Don't assume it's a bug yet—avoid using the word “bug” in the title until it's confirmed.
- Describe **expected behavior** vs **actual behavior**.
- Provide **reproduction steps** so someone else can recreate the issue. Isolate the problem when possible (e.g. minimal rule YAML, minimal config).
- Paste the information you collected above.

After you file:

- Maintainers will label the issue. If we can't reproduce it, we may ask for more steps and mark it needs-repro. Reproducible issues may be marked needs-fix and opened for implementation.

64.6 Suggesting Enhancements

Enhancements are tracked as GitHub issues.

64.6.1 Before submitting

- Use the latest version and read the documentation to see if the behavior already exists or can be configured.
- Search issues to see if the enhancement was already suggested; if so, add to that discussion.
- Consider whether the idea fits **this repository's scope**: the **open_fdd** rules engine (YAML rules, **RuleRunner**, **column_map**, expression cookbook patterns). Make a clear case for why the change would help most library users who run FDD on pandas.

64.6.2 How to submit a good enhancement suggestion

- Use a **clear, descriptive title**.

- Describe the **current behavior** and what you **want instead**, and why.
 - Provide **step-by-step detail** where useful; screenshots or example YAML/config can help.
 - Explain why this would be useful to the community.
-

64.7 Contributing Rules (Expression Rule Cookbook)

We especially welcome contributions from mechanical engineers and building professionals who can add or improve FDD rules.

- **Where rules live:** [Fault rules overview](#) — keep project rules in a **YAML directory** you pass to **RuleRunner**. Reload by constructing the runner again after disk edits.
- **How to write rules:** [Expression Rule Cookbook](#) — expression-type rules use YAML with BRICK-style inputs, params, and pandas/NumPy expressions. The cookbook includes AHU-style rules (e.g. GL36-inspired) and patterns you can adapt.
- **What to contribute:**
 - **New rule YAMLs** for common faults (AHU, VAV, plant, sensors) that others can reuse.
 - **Improvements to existing cookbook rules** (thresholds, logic, descriptions).
 - **New cookbook sections or examples** (e.g. for a specific equipment type or standard).

To contribute a rule or cookbook change:

1. Add or edit YAML under **examples/**, **open_fdd/tests/fixtures/rules/**, or your own **rules_dir** for new/updated rules; edit **docs/expression_rule_cookbook.md** (and related docs) for cookbook content.
 2. Follow the existing YAML style and BRICK naming used in the cookbook.
 3. In a PR, describe the fault the rule targets and any assumptions (e.g. Brick types, units). If it's from a standard (e.g. ASHRAE), note the reference.
-

64.8 Your First Code Contribution

- Look for issues labeled **good first issue** or **help wanted** (if we've added them).
- The codebase targets **Python 3.10+** (see `pyproject.toml`), with **pandas / NumPy** at the core. See [Getting started](#) and the README for setup.

- For rule or engine changes, run **pytest** and, when possible, a small **RuleRunner** smoke test on sample CSV data before submitting a PR.
-

64.9 Improving the Documentation

Documentation for **this repository** lives in docs/ and is published at **bbartling.github.io/open-fdd** (Just the Docs — **rules engine** focus). Improvements are welcome:

- Fix typos, clarify wording, or update steps.
- Add examples or how-tos (e.g. for the [Expression rule cookbook](#), [How-to guides](#), or [Column map resolvers](#)).
- Keep code blocks and links in sync with the codebase.

CI docs build: Pushes that touch docs/** run the **Docs (GitHub Pages)** workflow (on master/main) and the **docs** job in **CI** on PRs — both run `bundle exec jekyll build`. Preview with `bundle exec jekyll serve` under docs/ before merge.

Building a PDF: Run `python3 scripts/build_docs_pdf.py` (or `./scripts/build_docs_pdf.sh` after `pip install weasyprint pyyaml`). Requires [pandoc](#) and **weasyprint** (`pip install weasyprint`) or LaTeX (`--pdf-engine=pdflatex`). Output: `pdf/open-fdd-docs.pdf` and `pdf/open-fdd-docs.txt`.

CI / master: Pushes to master/main that change docs/** run the **Docs PDF** workflow. It opens **chore/docs-pdf-refresh** — merge that PR so the PDF bundle in pdf/ stays current (protected branch cannot take a direct bot commit). You can also run the workflow manually from Actions → Docs PDF → Run workflow.

Open a PR with your changes; for large edits, an issue first can help align with maintainers.

After changing README or high-traffic doc links, spot-check the live site (GitHub Pages: extensionless URLs like /`getting_started` work; a **trailing slash** on leaf pages can 404 depending on hosting).

```
BASE=https://bbartling.github.io/open-fdd
curl -sS -o /dev/null -w "%{http_code}" "$BASE/getting_started"
curl -sS -o /dev/null -w "%{http_code}" "$BASE/
column_map_resolvers"
curl -sS -o /dev/null -w "%{http_code}" "https://github.com/
bbartling/open-fdd/blob/master/pdf/open-fdd-docs.pdf" # expect
302 or 200
```

64.10 Styleguides

- **Python:** We use Black for formatting. Run black on changed files before committing.
 - **YAML:** Match the style in `examples/AHU/rules/` and the expression rule cookbook (indentation, key order).
 - **Markdown:** Use clear headings and lists; link to existing docs where relevant.
-

64.11 Git workflow (branch → PR → sync)

Create the branch **before** committing so that after the PR is merged, git pull on main/master fast-forwards and you avoid divergent-branch errors (protected main/master).

1. Create a branch

```
git checkout -b feature/short-name
```

2. Stage, commit, and push

```
git add .  
git commit -m 'short note describing the change'  
git push -u origin feature/short-name
```

3. Open a PR manually on GitHub from feature/short-name into main (or master). Create the PR in the GitHub UI; we do not use a CLI for this step.

4. After the PR is merged — fetch prune, switch to default branch, and pull to sync:

```
git fetch --prune  
git checkout main  
git pull
```

Use master instead of main if that is your default branch. If you ever committed on main/master before creating the branch, local and remote can diverge after the merge; run `git reset --hard origin/main` (or `origin/master`) to match the remote.

64.12 Commit Messages

- Use a clear, short summary line. Optionally add a body with details.

- Reference issues/PRs when relevant (e.g. Fix bounds rule when input is all NaN (#123)).
-

64.13 Join the Project Team

If you're interested in ongoing maintenance or a larger role, say so in an issue or reach out to the maintainers. We're especially interested in folks who can triage bugs, review rule contributions, or improve docs and the cookbook.

64.14 Attribution

This contributing guide was inspired by the [contributing.md](#) project. You don't need to pay to create or use a CONTRIBUTING file—it's just a markdown file in your repo that helps contributors and maintainers.

65 Expression cookbook (Python / Rule Lab)

66 Expression cookbook — Python (Rule Lab)

Production rules live in `workspace/data/rules_py/*.py`. Rule Lab saves metadata to `rules_store.json` and source to disk. Batch runner: `POST /api/rules/batch / fdd_runner`.

66.1 Rule shape

```
def evaluate(row, cfg, prev_row=None, rows=None):  
    """Return False, True, or (True, evidence_rows)."""  
    ...
```

Arg	Meaning
row	Current sample dict (ts, temp, value_column, value_kind, ...)
cfg	

Arg	Meaning
	Thresholds from rule config in store (per binding)
rows	History for windowed logic (flatline/spread)

Scripts (mode script): return out = {"df": dataframe} from playground — not scheduled as FDD.

66.2 Recipe 1 — Flatline 1 hour

Rolling spread of temp (or RH via value_kind) below tolerance:

```

from bench_fdd_common import hour_window_ready, window_rows_1h,
cfg_threshold, temp_unit_symbol

def evaluate(row, cfg, prev_row=None, rows=None):
    if rows is None:
        return False
    window_rows = window_rows_1h(row, rows)
    if not hour_window_ready(window_rows):
        return False
    tol_key = "flatline_tolerance_rh" if row.get("value_kind") ==
"rh" else "flatline_tolerance"
    vals = [r.get("temp") for r in window_rows if r.get("temp")
is not None]
    if not vals:
        return False
    spread = max(vals) - min(vals)
    if spread < cfg_threshold(cfg, tol_key):
        return True, window_rows
    return False

```

Copy from rules_py/flatline_1h.py. Tag rule with a **letter fault code** in Rule Lab (e.g. VAV-C for zone temp sensor fault — category sensor_fault).

Cookbook pattern	Typical fault codes	Rule file
flatline_1h	VAV-C, AHU-C, DC-C, BLD-B (OAT)	flatline_1h.py
spread_1h	VAV-B, AHU-A, CH-B	spread_1h.py, duct- t_spread_1h.py
oob_rolling	VAV-C (bounds), AHU-D	oob_rolling.py
custom_evaluate	AHU-B, VAV-A	site-specific rules_py

Browse the full map: GET /api/faults/graph or dashboard **Fault catalog**.

66.3 Recipe 2 — Spread / delta 1 hour

Max – min over 1 h above threshold — spread_1h.py, duct-t_spread_1h.py.

66.4 Recipe 3 — Out of bounds (rolling)

oob_rolling.py — compare sample to low / high in cfg.

66.5 Recipe 4 — Economizer / mixed air (site-specific)

Acme examples: acme_mixed_air_temp_oob_economizer_diagnostic.py, acme_zone_temp_out_of_bounds.py — bind to model fdd_input keys after BACnet import.

66.6 Config & bindings

Item	Where
Thresholds	Rule config in UI → passed as cfg
Point columns	Bindings → feather column names from BRICK fdd_input
Fault code	Letter suffix only (VAV-C, not VAV-03); must exist in GET /api/faults/catalog
Enable	Per-binding flag in rules_store.json

Test: **Lint** → **Test rule** in Rule Lab (POST /api/playground/test-rule).

66.7 Shared helpers

bench_fdd_common.py: window_rows_1h, hour_window_ready, cfg_threshold, temp_unit_symbol. Import in site rules; commit shared helpers to git.

Bench seed: python scripts/setup_bench_afdd.py.

66.8 AI drafting tips

- Start from flatline_1h.py or spread_1h.py; change thresholds and pick matching fault_code from the catalog (same cookbook_patterns row).
- Ask AI to explain **spread** and **window length** from test-rule output, not to skip batch run.
- bench-* rule ids are excluded from default scheduled batch — use duct-* or site prefixes for production timers.

See [Rule Lab storage](#).

67 Expression cookbook (YAML / pandas)

68 Expression cookbook — YAML / pandas

For `pip install open-fdd` and offline CSV work. Rules are YAML evaluated by **RuleRunner** on a pandas DataFrame. Not used by the edge scheduled loop (see [Python cookbook](#)).

68.1 Minimal expression rule

```
name: high_sat
type: expression
flag: high_sat_flag

inputs:
  sat:
    brick: Supply_Air_Temperature_Sensor
```

```

params:
    max_temp: 90.0

expression: |
    sat > max_temp

from open_fdd.engine import RuleRunner
runner = RuleRunner(rules_path="my_rules/")
df_out = runner.run(df,
column_map={"Supply_Air_Temperature_Sensor": "SAT"})

```

column_map: maps logical / Brick keys to DataFrame columns. Optional brick:, haystack:, dbo:, s223:, 223p: on inputs — first match wins. See [Column map resolvers](#).

68.2 Ontology labels (optional)

Brick/Haystack/DBO/223P keys on inputs are optional; plain logical names work.

68.3 Schedule + weather gates

```

params:
    schedule:
        weekdays: [0, 1, 2, 3, 4]
        start_hour: 8
        end_hour: 17
    weather_band:
        oat_input: Outside_Air_Temperature_Sensor
        units: imperial
        low: 32
        high: 85

expression: |
    fan_on & ~schedule_occupied & weather_allows_fdd

```

Injects `schedule_occupied` and `weather_allows_fdd` booleans into expression scope.

68.4 Signal scaling (0-1 vs 0-100 %)

BACnet often exposes 0-100. Expression rules do **not** auto-scale (unlike hunting / oa_fraction).

Use injected `normalize_cmd(series)` (open-fdd 2.3+):

```
expression: |
    (normalize_cmd(Supply_Fan_Speed_Command) >= drv_hi_frac -
drv_near_hi)
```

Or explicit: `np.where(cmd > 1, cmd / 100.0, cmd)`.

68.5 GL36-style example — duct static low at full fan

```
name: duct_static_low_at_full_speed
type: expression
flag: rule_a_flag

inputs:
    Supply_Air_Static_Pressure_Sensor:
        brick: Supply_Air_Static_Pressure_Sensor
    Supply_Air_Static_Pressure_Setpoint:
        brick: Supply_Air_Static_Pressure_Setpoint
    Supply_Fan_Speed_Command:
        brick: Supply_Fan_Speed_Command

params:
    sp_margin: 0.12
    drv_hi_frac: 0.93
    drv_near_hi: 0.06

expression: |
    (Supply_Air_Static_Pressure_Sensor <
Supply_Air_Static_Pressure_Setpoint - sp_margin) &
(normalize_cmd(Supply_Fan_Speed_Command) >= drv_hi_frac -
drv_near_hi)
```

More recipes (blend air bands, ERV, hunting): [examples/AHU/rules/](#), [Test bench catalog](#).

68.6 Built-in types (non-expression)

type	Purpose
bounds	Outside [low, high]
flatline	Rolling spread < tolerance
hunting	Excessive toggling
oa_fraction	OA fraction error
erv_efficiency	ERV effectiveness

Details: [Rules overview](#).

68.7 Validation (2.3+)

`RuleRunner.run(..., input_validation='strict')` fails fast on bad column maps. Production often uses `skip_missing_columns=True` with warnings.

68.8 Porting YAML → edge Python

When a rule graduates to production, reimplement window logic against feather **rows** in Rule Lab; bind via BRICK model. Keep YAML in repo for regression tests (`pytest open_fdd/tests/engine`).

69 Expression rule cookbooks

70 Expression rule cookbooks

Open-FDD uses **two rule surfaces**:

Cookbook	Runtime	Use when
<u>Python / Rule Lab</u>	Edge bridge, feather rows, <code>evaluate(row, cfg, ...)</code>	Production operator stack, Acme, bensserver PyPI, CSV
<u>YAML / pandas</u>	<code>open_fdd.engine.RuleRunner</code> on DataFrames	export, notebooks, CI fixtures

Pick one path per deployment. Edge **scheduled FDD** runs `workspace/data/rules_py/*.py`, not hot-reloaded YAML.

70.1 Quick links

- [Rule Lab storage](#)
- [Rules overview \(engine\)](#)
- [Test bench catalog](#)

71 Fault rules (engine / PyPI)

72 Fault rules (engine / PyPI)

YAML fault rules for `open_fdd.engine.RuleRunner` on pandas — **offline CSV, notebooks, and CI**. The **production edge** uses Python in Rule Lab instead ([Python cookbook](#)).

Page	Description
Overview	Rule types, RuleRunner
YAML expression cookbook	Expressions on DataFrames
YAML → pandas internals	Implementation notes
Test bench catalog	Fixture YAML

73 Modular Architecture (PyPI)

74 Modular architecture (PyPI)

The `open_fdd` package is a small library: **rules on pandas**, optional **reporting**. For edge deploy, see [System overview](#) and [Getting started](#).

74.1 Modules

Module	Role
<code>open_fdd.engine</code>	YAML rules, checks, RuleRunner , <code>column_map</code> resolvers
<code>open_fdd.reports</code>	Episode summaries, matplotlib plots, optional Word export

Module	Role
<code>open_fdd.schema</code>	pydantic fault result/event models (engine dependency)

74.2 Engine layers

Layer	Role
Rule YAML	Human-authored definitions loaded by <code>load_rule</code> / <code>RuleRunner</code> .
Column map	Logical names → DataFrame columns (dict, manifest, composite resolver).
Checks	Bounds, flatline, expression eval, schedule/weather masks, ...
Runner	Orchestrates checks and writes <code>*_flag</code> columns.

74.3 Reports (optional)

After `RuleRunner.run`, use `open_fdd.reports` for:

- `summarize_fault` / `summarize_all_faults` — duration and sensor stats during faults
- `get_fault_events` / `plot_fault_analytics` — events and charts ([reports] extra → matplotlib)
- `build_report` — .docx when `python-docx` is installed

74.4 See also

- [Engine API](#)
- [Reports API](#)
- [YAML expression cookbook](#)
- [Column map resolvers](#)

75 BACnet driver capabilities

76 BACnet driver capabilities

Open-FDD BACnet I/O lives in `bacnet_toolshed/` (BACpypes3) and two edge containers:

Component	Where	Role
<code>openfdd-commission</code>	<code>:8767</code>	Discovery jobs, read/write, priority-array, poll loop trigger
<code>openfdd-bacnet-poll</code>	host network	Scheduled RPM poll → workspace/bacnet/polls/samples.csv
<code>openfdd-bridge</code>	<code>:8765</code>	REST proxy, driver CSV registry, model import, feather ingest

Stack args come from `workspace/bacnet/commissioning/commission.env` (BACNET_BIND, ROUTER_IP, MSTP_NET, ...). See [Getting started — BACnet bind](#).

76.1 Summary matrix

BACnet capability	Status	How
Who-Is / I-Am	Supported	POST /api/bacnet/whois, CLI <code>discover_devices</code> , <code>commission agent</code>
Device discovery (inventory)	Supported	Who-Is → device address, vendor, description
Object-list read	Supported	Full list or segmented object-list if device aborts
Point discovery	Supported	

BACnet capability	Status	How
		Per device: OID + object-name + commandable flag
Full discover → CSV	Supported	Job writes points_discovered.csv (object-name, description, present-value, units)
ReadProperty	Supported	Any valid property_identifier on one object
ReadPropertyMultiple (RPM)	Supported	Chunked (≈25 props/request); poll uses RPM for present-value
Read priority-array	Supported	Per commandable point; full 16-slot decode
Supervisory / override scan	Supported	All commandable points → active priority slots
WriteProperty	Supported (gated)	Priority required ; value: null = release at that priority
WritePropertyMultiple	Not supported	Use repeated write or single RPM read
COV subscribe	Not supported	Use periodic poll
ReadRange	Not supported	—
ReinitializeDevice / DM	Not supported	—
File object / backup	Not supported	—
BBMD registration	Not built-in	Bind correct OT NIC/IP; use --route-aware for MSTP via IP router
Alarm / Event enrollment	Not supported	FDD uses historian + rules, not BACnet alarm ack

76.2 Discovery

76.2.1 Who-Is / device scan

- Instance range: DISCOVER_LOW ... DISCOVER_HIGH (env or API body).
- **BACnet/IP** broadcast on bound NIC.
- **MS/TP via router:** ROUTER_IP + MSTP_NET → Who-Is on network: *@router (see discover_lib.collect_i_ams).
- Returns I-Am fields: device identifier, address, description/object-name, max APDU, segmentation, vendor id.

76.2.2 Point discovery (one device)

POST /api/bacnet/point-discovery → reads **object-list**, RPM **object-name** for all points, RPM **priority-array** on commandable types (AO/AV/BO/BV/MSO/MSV/integer-value, ...) to set commandable: true/false.

76.2.3 Full discover job

POST /api/bacnet/discover (or commission POST /api/jobs/discover) runs CLI bacnet_toolshed.discover → **points_discovered.csv**.

Per-point properties during discover:

- object-name, description, present-value, units (best-effort; skips on error)

Bridge: GET /api/bacnet/inventory groups CSV by device.

76.3 Read path

Operation	API / CLI	Notes
Single property	POST /api/bacnet/read	Resolves device via cache or Who-Is
Multiple properties	POST /api/bacnet/read-multiple	List of (object_identifier, property_identifier)
Priority array	POST /api/bacnet/priority-array	All 16 slots with type + value
Supervisory check	POST /api/bacnet/supervisory-check	Override summary across commandable points

Encoding: atomic/sequence/array → JSON-friendly values
(bacnet_ops.encode_rpm_value).

76.4 Write path

Operation	API	Notes
Write present-value (or other prop)	POST /api/bacnet/write	property_identifier must be valid BACnet property id
Release (relinquish)	Same, value: null	Requires priority 1-16; writes BACnet NULL at that priority
Priority on write	Required	1 = manual override ... 16 = default

Safety gates (bridge):

- OFDD_ENABLE_BACNET_WRITE=1 or writes return **403**
- Optional workspace/bacnet/write_allowlist.json — device_instances and/or object_identifiers
- Role **integrator** for write; commission/operator for reads
- Audit log on denied writes

Not supported: write without priority, WritePropertyMultiple, confirmed COV, out-of-service manipulation APIs.

76.5 Poll driver (historian)

Item	Detail
Property	present-value only (RPM per device)
Intervals	60, 300, 600, 900 seconds per point (points.csv poll_interval_s)
Output	workspace/bacnet/polls/samples.csv (long format)
Ingest	POST /api/bacnet/poll/once or internal ingest → feather wide frames (Arrow IPC; compaction deferred — see Arrow data plane)
Network	Poll container network_mode: host (same UDP bind semantics as commission)

Enable points: `bacnet_toolshed.enable_points` or bridge PATCH `/api/bacnet/driver/point`, merge-rows, driver tree UI.

76.6 Point mapping & BRICK model

Step	Mechanism
Discovered rows	<code>points_discovered.csv</code> — BACnet oid, names, optional <code>brick_class</code> / <code>brick_tag</code> columns
Poll list	<code>points.csv</code> — <code>enabled=1</code> , <code>point_id</code> , <code>series_id</code> , site/building/system ids
Driver registry	Bridge GET <code>/api/bacnet/driver/tree</code> — <code>devices/points</code> , poll toggles, remap
Model merge	POST <code>/api/bacnet/import-to-model</code> , POST <code>/api/bacnet/driver/sync-discovery</code>
Rule bindings	<code>fdd_input</code> on BRICK points → Rule Lab <code>column_map</code> at FDD run

Driver maintenance APIs: merge rows, enable/disable poll per point or device, remap instance/address, delete point/device, clear registry (+ model sync).

76.7 Local BACnet server points (edge identity)

Commission agent exposes read-only **BV/AV** on the edge device instance (`server_points.py`): e.g. edge online, commission OK, poll row count, devices discovered — for BACnet supervisors that read the head-end.

GET `/api/bacnet/server/points` returns the snapshot.

76.8 Network / bind checklist

Setting	Purpose
<code>BACNET_BIND</code>	<code>ip/mask:47808</code> — OT interface (required)
<code>BACNET_NAME</code> / <code>BACNET_INSTANCE</code>	Local virtual device
<code>ROUTER_IP</code> + <code>MSTP_NET</code> + <code>--route-aware</code>	MS/TP behind IP router
<code>DISCOVER_TIMEOUT</code>	

Setting	Purpose
	Who-Is / discover wait (seconds)
Host network poll	Poll image must share host routing to reach device_address

CLI mirrors env: `python -m bacnet_toolshed.discover ... --address 192.168.x.x/24:47808.`

76.9 REST quick reference

Full paths and auth: [Bridge API — BACnet](#).

Commission-only jobs: GET /api/bacnet/jobs/{job_id} after discover / point-discovery / supervisory-check (202 + job id).

76.10 Related

- [BACnet toolshed](#) — CLI modules and workspace paths
- [bacnet_toolshed/README.md](#) — command examples
- [BRICK + BACnet modeling](#)
- [Security hardening](#) — write gates and auth

77 BACnet

78 BACnet

BACpypes3 **commissioning**, **polling**, and **bridge APIs** for field BACnet/IP and MS/TP (via IP router). The PyPI open-fdd engine does not include BACnet — only this repo checkout.

Doc	Content
Driver capabilities	Supported vs not — discover, read, RPM, write, release, priority-array, poll, mapping
BRICK + BACnet	Model import and fdd_input
Getting started — bind	BACNET_BIND, bench overlay
Bridge API — BACnet	REST table

Repo: bacnet_toolshed/README.md for CLI (discover, enable_points, poll_driver, commission_agent).

79 API reference (index)

80 API reference

API	Audience	Doc
Operator Bridge (REST)	Integrators, dashboard, deployment AI	Bridge API
<code>open_fdd.engine</code>	Offline YAML rules on pandas	Engine API
<code>open_fdd.reports</code>	Fault summaries and plots	Reports API
<code>open_fdd.schema</code>	Result/event models	Used by engine; see engine doc

Production buildings use the **bridge** + Rule Lab Python. PyPI engine is for CSV/notebook and CI regression.

81 Operator Bridge API

82 Operator Bridge API

REST API served by **openfdd-bridge** (default `http://127.0.0.1:8765`). Production: Caddy terminates TLS/HTTP on `:80` and proxies to the bridge.

OpenAPI: GET `/docs` and GET `/redoc` when the bridge runs.

Auth: JWT via POST `/api/auth/login`. Most routes require Authorization: Bearer `<token>`. Roles: viewer, operator, integrator, commission, agent, admin. BACnet writes require commission + [write guard](#). Dev: `OFDD_AUTH_DISABLED` on trusted localhost only.

WebSocket: live dashboard refresh — pass `?token=` when auth enabled.

82.1 Health & audit

Method	Path	Role	Description
GET	/health	public*	Liveness, poll worker hint
GET	/health/stack	auth	Compose-level probes (model SPARQL, Ollama, MCP)
GET	/api/audit/summary	auth	Audit counters
GET	/api/audit/events	auth	Audit log (query params)
GET	/api/audit/errors	auth	Recent API errors

*Exact public set depends on `OFDD_AUTH_DISABLED` and middleware.

82.2 Auth

Method	Path	Description
POST	/api/auth/login	Username/password → JWT
GET	/api/auth/me	Current user
GET	/api/auth/status	Auth mode / lockout hints

82.3 Rules & FDD

Method	Path	Role	Description
GET	/api/rules/saved	user	Rule index
POST	/api/rules/save	operator+	Create/update rule metadata
GET	/api/rules/saved/{id}/source	user	Python source
PUT	/api/rules/saved/{id}/source	operator+	Write .py file
GET		user	Point bindings

Method	Path	Role	Description
	/api/rules/assignments		
POST	/api/rules/bind	integrator+	Bind rule to point
POST	/api/rules/bindings	integrator+	Bulk bindings
POST	/api/rules/batch	operator+	Run all enabled rules → fdd_results.json
GET/POST	/api/rules/drafts	user	Draft rules

82.4 Rule Lab playground

Method	Path	Description
POST	/api/playground/lint	AST lint Python rule
POST	/api/playground/test-rule	Run evaluate() on feather window
POST	/api/playground/run-script	Execute script mode
GET	/api/playground/sample-frame	Sample DataFrame for editor

82.5 BRICK / data model

Method	Path	Description
GET	/api/model/sites	Site list
GET	/api/model/tree	Equipment tree (SPARQL)
GET	/api/model/graph	RDF graph fragment
GET	/api/model/scope	Scoped query
GET	/api/model/export	Export TTL/JSON
GET	/api/model/health	Model point/equipment counts
GET	/api/model/ttl	TTL metadata
POST	/api/model/sync-ttl	Regenerate TTL from JSON

Method	Path	Description
POST	/api/model/import	Import model payload
POST	/api/model/sites	Create site
GET/ POST	/api/model/bacnet- sync	BACnet ↔ model sync state

82.6 BACnet

Method	Path	Role	Description
GET	/config/ bacnet	read	Commission URL / config
GET	/api/ bacnet/ commission/ status	read	Commission agent health
GET	/api/ bacnet/ server/ points	read	Server point table
GET	/api/ bacnet/ inventory	read	Discovered inventory
POST	/api/ bacnet/ whois	commission	Who-Is
POST	/api/ bacnet/ discover	commission	Discover devices
POST	/api/ bacnet/read	commission	Read property
POST	/api/ bacnet/ read- multiple	commission	RPM read
POST	/api/ bacnet/ write	write	Supervisory write (gated)
POST	/api/ bacnet/ import-to- model	integrator	Inventory → BRICK
POST	/api/ bacnet/ driver/	commission	Sync driver tree

Method	Path	Role	Description
	sync-discovery		
POST	/api/bacnet/poll/once	commission	Trigger poll cycle
GET	/api/bacnet/poll/status	read	Poll worker status
POST	/ingest/bacnet	read	Ingest samples into feather

Bind address: BACNET_BIND in commission env (see [Getting started](#)).
Feature matrix: [BACnet driver capabilities](#).

PATCH | /api/bacnet/driver/point | commission | Enable poll + interval on point |
PATCH | /api/bacnet/driver/device | commission | Enable poll for all points on device |
PATCH | /api/bacnet/driver/device/remap | commission | Change instance and/or address |
DELETE | /api/bacnet/driver/point/{point_id} | commission | Remove point from registry |
DELETE | /api/bacnet/driver/device/{device_instance} | commission | Remove device |
DELETE | /api/bacnet/driver/registry | commission | Clear CSVs + model sync |

82.7 Faults (check-engine)

Method	Path	Description
GET	/api/faults/catalog	Fixed fault codes
GET	/api/faults/status	GREEN/YELLOW/RED aggregate
GET	/api/faults/tree	Fault tree by equipment
GET	/api/faults/code/{code}	Code metadata

82.8 Timeseries & sites

Method	Path	Description
GET	/api/timeseries/sites	Feather sites
GET	/api/timeseries/series	Series metadata

Method	Path	Description
GET	/api/timeseries/plot	Plot payload
GET	/api/timeseries/readings	Raw readings
GET	/api/sites	Site list
GET	/api/sites/{site_id}/frame	Wide frame for site

82.9 Building & host

Method	Path	Description
GET	/api/building/status	Building summary
GET	/api/building/alerts	Active alerts
GET	/api/host/stats	CPU/mem/disk

82.10 Local agent (Ollama)

Prefix `/openfdd-agent`. Role **agent** for tool execution.

Method	Path	Description
GET	/openfdd-agent/context	Composed prompt context
GET	/openfdd-agent/tools	Tool manifest
POST	/openfdd-agent/tool	Execute tool (e.g. rules.save)
GET	/openfdd-agent/ollama/health	Ollama reachability
GET	/openfdd-agent/building-insight	Check-engine narrative context
GET	/openfdd-agent/zone-temps	Zone temperature insight
POST	/openfdd-agent/chat	Chat completion (catalog-bound)

See [Local Ollama](#).

82.11 Modbus

Method	Path	Description
POST	/api/modbus/read_registers	Read holding registers

82.12 PyPI engine API (separate)

Library-only HTTP is **not** bundled. See [Engine API](#) and [Reports API](#) for RuleRunner and open_fdd.reports.

82.13 Errors

JSON error bodies; stack traces hidden unless OFDD_DEBUG_TRACEBACKS=1. See [Security hardening](#).

83 Appendix

84 Appendix

Integrator and maintainer reference.

Page	Description
Operator Bridge API	REST routes (auth, BACnet, rules, faults, agent)
BACnet driver capabilities	What discovery, read, write, poll, and mapping cover
API reference (index)	Bridge vs PyPI engine
Engine API	RuleRunner, YAML loaders
Reports API	Summaries, plots, .docx
Technical reference	Repo layout, tests
Developer guide	Contributing to open_fdd

85 Overview

86 Fault rules

Fault rules are **YAML-defined** checks run against **pandas** DataFrames. Each rule produces integer fault flag columns (0 / 1, and related outputs) via **RuleRunner**.

Each rule declares **logical input names** in YAML. You supply a **column_map** from those names (or optional ontology keys like **Brick** class labels) to your DataFrame columns. See [Column map resolvers](#) and the [YAML expression cookbook](#).

86.1 Where rules live

Keep rule YAML in a **single directory** (for example `my_rules/`) or pass parsed dicts to **RuleRunner(rules=[...])**. Example snippets live under **examples/** and **open_fdd/tests/fixtures/rules/** in this repository.

See the [YAML expression cookbook](#) to add or adapt rules. For how YAML becomes pandas operations, see [YAML rules → Pandas \(under the hood\)](#).

86.2 Hot reload

RuleRunner reads the rule list you give it at construction time. To pick up disk edits, call **load_rules_from_dir** again (or construct a new **RuleRunner**) before the next **run()**.

86.3 Rule types

Type	Purpose
bounds	Value outside [low, high]
flatline	Rolling spread < tolerance (stuck sensor)
hunting	Excessive state changes (PID hunting)
expression	Custom pandas/numpy expression; supports schedule/weather gating via

Type	Purpose
	params.schedule and params.weather_band
oa_fraction	OA fraction vs design airflow error
erv_efficiency	ERV effectiveness out of range

These six values are the built-in type values in the engine.

Occupied-hours and weather gating are not separate rule types; they are optional masks injected for type: expression rules (schedule_occupied, weather_allows_fdd). See the [YAML expression cookbook](#).

86.4 YAML structure

Example with Brick-style logical keys (map them with `column_map`):

```
name: sensor_bounds
type: bounds
flag: bad_sensor
inputs:
  Outside_Air_Temperature_Sensor:
    brick: Outside_Air_Temperature_Sensor
  Supply_Air_Temperature_Sensor:
    brick: Supply_Air_Temperature_Sensor
params:
  low: 40
  high: 90
  rolling_window: 6  # optional: require N consecutive True
                      # samples before flagging
```

Per-rule rolling window: Set `params.rolling_window` (e.g. 6) in a rule to require that many consecutive True samples before the fault is flagged. Omit for “flag on any True.” See fixtures such as `sensor_flatline.yaml` under `open_fdd/tests/fixtures/rules/`.

86.5 Running rules

```
from pathlib import Path
from open_fdd.engine.runner import RuleRunner

runner = RuleRunner(rules_path=Path("path/to/rules"))
out = runner.run(df, timestamp_col="timestamp", column_map={...})
```

86.6 Engineering metadata and analytics

Downstream analytics often combine **fault flags** from **RuleRunner** with **equipment metadata** and **time series** you already store. Use **column_map** to align ontology or vendor labels with DataFrame columns — see [Column maps & ontologies](#) and [Column map resolvers](#).

86.7 Cookbook

See [YAML expression cookbook](#) and `examples/AHU/rules/` for GL36-style recipes.

87 Test bench rule catalog

88 Rule YAML catalog (in this repository)

Example and test rules live next to the engine sources. Use them as templates for your own `rules_dir`.

88.1 `open_fdd/tests/fixtures/rules/` (pytest)

File	Purpose
<code>weather_gust_lt_wind.yaml</code>	Weather / wind gust check
<code>weather_rh_out_of_range.yaml</code>	RH bounds
<code>weather_temp_spike.yaml</code>	Temperature spike
<code>weather_temp_stuck.yaml</code>	Temperature stuck
<code>test_flatline_sat.yaml</code>	Flatline SAT
<code>test_sensor_flatline_1774618711.yaml</code>	Sensor flatline test

88.2 examples/AHU/rules/ (notebooks / demos)

File	Purpose
sensor_bounds.yaml	Bounds example
sensor_flatline.yaml	Flatline example
sat_operating_band.yaml	SAT band

88.3 Related docs

- [Fault rules overview](#)
- [Expression rule cookbook](#)

89 YAML rules → Pandas (under the hood)

90 YAML rules → Pandas DataFrames (under the hood)

This note is for engineers who want to see **exactly** how Open-FDD turns **rule YAML on disk** into **pandas operations** on **time-series DataFrames**—and where Brick TTL fits in. It complements [Fault rules overview](#) and [standalone CSV / pandas](#).

90.1 1. YAML never becomes “one giant DataFrame of rules”

Rule files are **configuration**, not tabular data. Open-FDD:

1. Reads each *.yaml in rules_dir with PyYAML (yaml.safe_load) into a **Python dict** per file (open_fdd.engine.runner.load_rule / load_rules_from_dir).
2. Keeps those dicts in a **list[dict]** inside RuleRunner (RuleRunner(rules=...) or RuleRunner(rules_path=...)).
3. At evaluation time, walks that list and, for each rule, computes a **boolean mask** (pandas.Series) aligned to the **sensor**

DataFrame's index, then writes a **new column** for the fault flag (e.g. `my_rule_flag`) as **integer 0 / 1**.

So: **rules = list of dicts in memory; data = one wide-ish DataFrame** of timestamps and point columns.

90.2 2. From long telemetry rows to a wide DataFrame

A common pattern (warehouse export, historian CSV, SQL query) is a **long** table (timestamp, point_id, value). For **RuleRunner** you typically:

1. Load rows into pandas.
2. **Pivot to wide**: `df.pivot_table(index="timestamp", columns="point_id", values="value")` (or equivalent) so each sensor is a column.
3. **Rename columns** using `column_map` (dict or manifest) so rule inputs line up with Brick-style or logical keys.
4. Parse the index with `pd.to_datetime` when you need time-based checks.

pandas handles aggregation during the pivot when duplicate index/column pairs exist; the result is one row per timestamp.

90.3 3. What RuleRunner.run does to that DataFrame

`RuleRunner.run(open_fdd.engine.runner):`

1. `result = df.copy()` — rules never mutate the caller's frame in place (callers can still hold a reference to the original).
2. For **each** rule dict:
 - Derives `flag_name` from `flag` or `{name}_flag`.
 - Calls `_evaluate_rule` → returns a **boolean Series** (fault mask) aligned to `result.index`.
 - Optionally applies a **rolling window** on the mask:
`mask.astype(int).rolling(window=rw).sum() >= rw` so a fault must persist for N consecutive samples.
 - Assigns `result[flag_name] = ...` as integer 0/1.

So each rule adds **one column** of flags; the frame grows **width-wise**, not row-wise.

90.4 4. How a YAML rule dict becomes pandas expressions

Inside `_evaluate_rule`, Open-FDD branches on `rule["type"]` (e.g. `bounds`, `flatline`, `expression`, ...):

- **column_map resolution:** For each logical input key, the runner picks a **DataFrame column name**. If the YAML input has a **brick** class, the supplied **column_map** dict is consulted (brick class key → column label), so the **same YAML** can run against different exports as long as the map matches your frame.
- **Bounds / thresholds:** Typical pattern is `(series < low) | (series > high)` using **vectorized** comparisons — these are **numpy ufuncs** under pandas, no Python for over rows.
- **Expressions:** String expressions may be evaluated in a **restricted eval** context (`open_fdd.engine.checks.check_expression`) with named series bound to `result[col]` — still vectorized.

If a required column is missing, the runner either **raises** or **skips** the rule (`skip_missing_columns=True`), depending on the call site.

90.5 5. Reloading rules vs DataFrame lifetime

If you call `load_rules_from_dir` before each **RuleRunner** construction, edits to YAML on disk take effect on the **next** run. The **DataFrame** you pass in exists only for that evaluation; construct a fresh frame for each batch or window as your pipeline requires.

90.6 6. Mental model checklist

Artifact	In-memory shape	pandas role
Rule YAML files	N/A (on disk)	None until loaded
Loaded rules	<code>list[dict]</code>	None
Telemetry window	<code>DataFrame</code> (time × points)	<code>pivot</code> , <code>datetime index</code> , <code>column rename</code>
Rule output	Same index, +flag columns	<code>vectorized masks</code> , <code>optional rolling</code>
Downstream store	Your app persists outputs if needed	After flags are computed

For a **minimal** example of rules + CSV (no DB), see [standalone CSV / pandas](#) and unit tests in `open_fdd/tests/engine/test_runner.py`.

91 Skills and agent shell

92 Skills and agent shell

The repository ships the **pandas rules engine** on PyPI. Dashboards, HTTP bridges, ingest drivers, MCP retrieval, and deployment recipes live under **skills/** and are built on demand with `openfdd.toml` plus the local **agent shell**.

92.1 Install paths

Goal	Command
Bare import / DataFrame work	<code>pip install open-fdd</code>
YAML rules + RuleRunner	<code>pip install "open-fdd[engine]"</code>
Maintainer checkout	<code>pip install -e "[dev]"</code>
Agent shell (not on PyPI)	<code>pip install -e packages/openfdd-agent-shell</code>

The `[engine]` extra adds **PyYAML** and **pydantic** for rule loading and validation. NumPy arrives transitively through **pandas**.

92.2 Manifest

Copy `openfdd.toml.example` to `openfdd.toml` and set:

- `[build].targets` — `api`, `dashboard`, `feather_storage`, ...
- `[build].drivers` — `csv`, `openmeteo`, `bacnet`, ...
- `[build].auth` / `[build].deploy` — `local`, `Caddy`, `systemd`, `Ansible` `bench`
- `[agent].skills` — skill folder names under **skills/**
- `[memory]` — `MEMORY.md` bootstrap path, daily note lookback, truncation budget, and `working-divergence.md` tail for skill/spec drift
- `[cron]` — `jobs.json`, runtime state, and run log directories

Generated application code belongs in **workspace/** (see **AGENTS.md**). Portfolio memory and schedules live beside generated services under the same workspace tree.

Path safety: at load time, `Manifest.load` resolves `agent.scratch_dir`, all `[memory]` paths, `[cron]` job/state/run paths, and `[wake]` lock/debounce/wake-log paths to absolute paths and **rejects** any entry that would resolve **outside** `workspace_dir`. Keep custom paths under `workspace/` (for example `workspace/scratch/`, `workspace/cron/`, `workspace/memory/`).

92.3 Agent shell

```
openfdd-agent-shell --repo-root .
```

The shell loads **AGENTS.md**, workspace **MEMORY.md** (plus recent daily notes and the architecture divergence log), selected **skills/*/SKILL.md** files, and launches **Codex CLI** with a composed system prompt. Slash commands include `/skills`, `/plan`, `/verify`, `/engine-check`, `/open-workspace`, `/memory`, and `/cron`.

When working code under **workspace/** diverges from skills because the documented path failed, append to `workspace/memory/architecture/working-divergence.md`. Scheduled `codex_turn` jobs can set `payload.wake_mode` to `mini` or `critique` to repeat the BAS-style append/triage loop.

Workspace cron CLI:

```
openfdd-workspace-cron --repo-root . list
openfdd-workspace-cron --repo-root . tick
```

Scheduled wake (mini + critique, transcript under `workspace/cron/wakes/`):

```
openfdd-wake --repo-root . --dry-run
openfdd-agent-shell wake --repo-root . --dry-run
```

Dry-run a single turn:

```
openfdd-agent-shell --repo-root . --dry-run --message "scaffold
csv ingest only"
```

The interactive REPL stays open after a Codex turn (a successful `run_invocation` does not exit the shell).

92.3.1 Workspace cron

`openfdd-workspace-cron add` requires **exactly one** schedule flag: `--every-seconds`, `--at` (ISO timestamp), or `--cron` (expression). `--payload-json` must be valid JSON (invalid JSON exits with a clear message, not a traceback).

Schedule objects loaded from `jobs.json` are validated: `every` needs `every_seconds > 0`; `cron` needs a non-empty `cron_expr`; `at` needs `at_iso`.

`tick` and `run --dry-run` advance job state like real runs. A **dry-run** execution clears the running flag, updates `next_run_at`, and writes a run record so jobs are not stuck re-queued.

Shell commands in cron job payloads are tokenized with `shlex.split` before `subprocess.run` (`shell=False`).

Example `codex_turn` job payload for wake-style turns:

```
{
  "wake_mode": "mini",
  "invocation": 1,
  "total": 2
}
```

Use `"wake_mode": "critique"` for the critique-only message. Non-numeric `invocation` / `total` values fall back to manifest defaults instead of raising.

92.3.2 Scheduled wake (mini + critique)

`openfdd-wake` (alias `openfdd-agent-shell wake`) runs up to `wake.mini_invocations` mini Codex turns, then one critique turn. Transcripts live under `workspace/cron/wakes/`.

Outcome	Behavior
Codex exit 0	Debounce timestamp updated; daily note records wake complete
Codex exit non-zero	Wake marked failed in the log; debounce not updated; critique skipped if a mini run failed
<code>--dry-run</code>	Prints codex exec lines only; no subprocess
Lock busy	<code>WakeLockError</code> ; another wake may be running
Debounce window	Skips run when <code>min_minutes_between</code> not elapsed

Wake lock files use a stale timeout (default six hours) so crashed runs can recover.

92.3.3 Memory layout

Path	Role
workspace/MEMORY.md	Bootstrap facts (truncated to bootstrap_max_chars)
workspace/memory/daily/YYYY-MM-DD.md	Session notes
workspace/memory/<category>/<entity>.md	Domain notes (category is a single safe segment; no .. or path separators)
workspace/memory/architecture/working-divergence.md	Skill vs workspace drift log

truncate_bootstrap never returns more characters than the configured maximum (including very small limits).

92.4 BACnet toolshed (repo checkout)

Field BACnet discovery, commissioning CSVs, and polling are in **bacnet_toolshed/** (not on PyPI). See **BACnet toolshed** and `openfdd.toml.example [build].drivers / driver-bacnet-ingest skill`.

92.5 Rule authoring (operator stack)

Python rules for **Rule Lab** live under `workspace/data/rules_py/` with metadata in `rules_store.json`. Humans edit in `/rule-lab`; the **agent** role can write the same files via `POST /openfdd-agent/tool (rules.save)`. See **Rule Lab — Python storage**.

Expression and YAML semantics for the **library** (`pip install "open-fdd[engine]"`) are unchanged. `RuleRunner.run` adds **integer** fault flag columns (0 / 1), not booleans—treat them as ints in pandas and downstream code. Missing input columns **fail** the run by default; `per-rule skip_missing_columns: true` skips checks for absent columns.

For edge Python rules: **Python expression cookbook**. For PyPI YAML: **YAML cookbook** and **Rules overview**.

92.6 Tests (CI)

From the repo root:

```
pip install -e ".[dev]"
pip install -e "packages/openfdd-agent-shell[dev]"
pytest open_fdd/tests/engine -q
pytest packages/openfdd-agent-shell -q
```

The **agent-shell** job in `.github/workflows/ci.yml` runs the agent-shell suite on Python 3.12.

92.7 Operator dashboard (committed starter)

A full **Rule Lab** stack ships under `workspace/api` and `workspace/dashboard` — see **Operator dashboard (Rule Lab)**. Agents should extend that code rather than scaffold from zero.

92.8 Bridge and MCP (generated under workspace/)

When the agent scaffolds a FastAPI bridge and React dashboard (skills `fastapi-bridge-api`, `react-operator-dashboard`, `codex-agent-on-bridge`), the browser calls **POST /openfdd-agent/chat** on the bridge host—not Codex directly. Operator retrieval and route hints: **Agent & operator playbook**.

Historical desktop/MCP how-tos under `howto/desktop_app.md` describe the retired monolith; new integrations should follow `skills/` and `workspace/`.

93 Operator dashboard (Rule Lab)

94 Operator dashboard (Rule Lab)

The **operator stack** lives under `workspace/`. Python runs on the **bridge host** (pandas, NumPy, sandboxed Rule Lab code); the browser is a compiled React SPA (same artifact Ansible ships to edge hosts).

94.1 Layout

Path	Role
workspace/api/	FastAPI bridge (openfdd_bridge) — API on 8765
workspace/api/static/app/	Production React build (served by bridge)
workspace/dashboard/	React source — build only; not served at runtime on edge
workspace/data/	Feather store, rules, model JSON
workspace/deploy/	Caddy + legacy systemd unit examples (non-Docker edges)
bacnet_toolshed/	BACnet commission agent + poll loop

94.2 Quick start (recommended — Docker)

```
pip install -e ".[dev]"
cp workspace/auth.env.example workspace/auth.env.local #
optional

./scripts/build_and_test.sh # vitest + prod UI + pytest
(first time)
./scripts/docker_build.sh # once per image change
./scripts/openfdd_stack.sh up # bridge + commission + mcp-
rag on :8765
```

Open <http://127.0.0.1:8765/>. On field VMs, Caddy serves <http://<lan-ip>/> (see [Edge deploy \(Docker\)](#)).

BACnet lab (bensserver): `docker compose -f docker/compose.dev.yml -f docker/compose.bench.yml up -d` and `./scripts/setup_local_testbench.sh`.

94.2.1 Legacy quick start (systemd on host)

```
./scripts/run_local.sh restart # venv + systemd units +
optional Caddy
```

`run_local.sh` rebuilds the production UI each restart unless `--ui-skip`.

94.2.2 UI build modes

Command	What it does
<code>./scripts/run_local.sh restart</code>	Production vite build (default)
<code>./scripts/run_local.sh restart --ui-test</code>	vitest run + production build
<code>./scripts/run_local.sh restart --ui-skip</code>	Skip npm; use existing static/app/
<code>./scripts/run_local.sh start --dev</code>	Also Vite on 5173 (HMR only — not edge parity)

CI / pre-deploy gate:

```
./scripts/build_and_test.sh # vitest + prod build + pytest
```

94.3 Manual dev (two terminals)

For rapid UI iteration without rebuilding:

```
# Terminal 1 – API
export OPENFDD_REPO_ROOT="$(pwd)"
export OFDD_DESKTOP_DATA_DIR="$PWD/workspace/data"
cd workspace/api && uvicorn openfdd_bridge.main:app --reload --port 8765
```

```
# Terminal 2 – Vite dev (proxies /api to 8765)
cd workspace/dashboard && npm ci && npm run dev
```

Open `http://127.0.0.1:5173`. For **Ansible/production parity**, use `run_local.sh` without `--dev` and hit the Caddy or `:8765` compiled URL.

94.4 Production build only

```
./scripts/build_operator_dashboard.sh prod # or: test (vitest + build)
# Bridge serves workspace/api/static/app/
```

94.5 Tabs (auth may be required)

Route	Page
/	Building check-engine (public)
/faults	Fault catalog (public)
/rule-lab	Python Rule Lab
/data-model	BRICK model + rule mapping
/plot	Feather trend plots
/bacnet	BACnet commissioning
/agent	Local Ollama chat
/host	CPU/RAM charts + data-disk space

94.6 Rule Lab modes

1. **Per-row rule** — `evaluate(row, cfg, ...)` → `POST /api/playground/test-rule`
2. **DataFrame script** — `out = {"df": ...}` → `POST /api/playground/run-script`

Saved rules use a **dual-file** layout under `workspace/data/`:

- **rules_store.json** — metadata, config, bindings, `fault_code`, `source_path`
- **rules_py/*.py** — canonical Python (always written on save; this is what `fdd_runner` executes)

The Rule Lab editor loads/saves via the bridge API; the footer shows the `.py` path. Bindings (which BACnet points a rule uses) are set on `/data-model`, not Rule Lab.

Full detail: Rule Lab — Python storage & shared editing (browser flow, AI tools, scheduled loop, BACnet → feather pipeline).

Schedule batch runs: `POST /api/rules/batch`, edge `openfdd-fdd-loop.timer` (Docker: `exec` into bridge), or local docker `compose exec bridge python -m openfdd_bridge.fdd_runner --once`.

94.7 Auth (OT LAN)

Copy `workspace/auth.env.example` → `auth.env.local`. When `OFDD_AUTH_SECRET` is set, protected routes require login.

See `workspace/deploy/README.md`, `workspace/deploy/SECURITY.md`.

94.8 AI maintainers

Skills: **fastapi-bridge-api**, **react-operator-dashboard**, **rules-crud-and-batch-run**.

- Rule storage & shared human/AI editing: [Rule Lab — Python storage](#)
- Ollama chat: `POST /openfdd-agent/chat`; rule writes: `POST /openfdd-agent/tool (rules.save)` with **agent** role

See also [Skills and agent shell](#) and [BACnet toolshed](#).

95 Rule Lab — Python storage & shared editing

96 Rule Lab — Python storage & shared editing

Operator FDD rules are **Python**, not hot-reloaded YAML. Humans edit them in the **Rule Lab** browser tab; the bridge API and scheduled runner read the **same files on disk**. An AI assistant with the **agent** role can write the same sources via `POST /openfdd-agent/tool (rules.save)`.

96.1 Where files live

Data root: **workspace/data/** (override with `OFDD_DESKTOP_DATA_DIR`).

Path	What it holds
<code>workspace/data/rules_store.json</code>	Rule index : id, name, mode (rule script), config, bindings, applies_to, fault_code, severity, enabled, source_path
<code>workspace/data/rules_py/*.py</code>	Canonical Python source — one file per saved rule (slug from rule name)
<code>workspace/data/fdd_results.json</code>	Latest batch FDD output → building check-engine light
<code>workspace/data/feather_store/</code>	Timeseries the rules run against (BACnet poll, CSV, demo)

Path	What it holds
workspace/data/ model.json	BRICK model — point fdd_input keys feed column_map at run time

Dual-write model: saving always updates **both** JSON metadata and a .py file. At run time, code is read from disk first (source_path); inline code in JSON is a fallback only.

Implementation: openfdd_bridge.rule_store,
openfdd_bridge.rule_source, openfdd_bridge.fdd_runner.

96.1.1 Git

- rules_store.json is **gitignored** (site-specific bindings and enable flags).
- rules_py/*.py is **tracked** — commit cookbook/bench templates; operators can version-control rule logic without the JSON index.
- Bench seed: python scripts/setup_bench_afdd.py repopulates store + .py files for local demos.

96.2 Browser (Rule Lab tab)

Route: /rule-lab (RuleLabPage.tsx + PythonCodeEditor).

96.2.1 What you do in the UI

1. **Pick or create** a rule from the saved-rules dropdown.
2. **Edit Python** in the editor (mode **rule**: evaluate(row, cfg, ...); mode **script**: DataFrame out = {"df": ...}).
3. **Lint** (debounced) → POST /api/playground/lint.
4. **Test** on live/demo feather data → POST /api/playground/test-rule or POST /api/playground/run-script.
5. **Save** → writes .py + updates rules_store.json.
6. **Update all records** → POST /api/rules/batch (same as scheduled FDD).
7. The footer shows the on-disk path, e.g. File: workspace/data/rules_py/bench_oa-t_flatline_1h.py.

96.2.2 Save flow (existing rule)

PUT /api/rules/saved/{id}/source ← editor text → .py file
 POST /api/rules/save ← metadata, config,
 fault_code, bindings ref

New rules use POST /api/rules/save only (creates id + .py in one step).

96.2.3 Bindings (not on Rule Lab)

Map rules to BACnet points on /data-model (Rule mapping board):

- POST /api/rules/bindings — point_ids, equipment_ids, brick_types
- At batch run, the bridge merges point fdd_input / BRICK keys into column_map.

96.2.4 Auth

Role	Rule Lab
integrator	Full edit, save, batch
agent	Same + POST /openfdd-agent/tool
operator	Read-only warning — view code, no save

96.3 Scheduled FDD (same Python)

On Docker edges, openfdd-fdd-loop.timer runs fdd_runner inside the bridge container. Local dev (./scripts/openfdd_stack.sh up) uses the commission poll loop; trigger FDD manually with docker compose exec bridge python -m openfdd_bridge.fdd_runner --once or POST /api/rules/batch.

Legacy local stack (./scripts/run_local.sh start) starts a background loop:

```
# workspace/api — required working directory
python -m openfdd_bridge.fdd_runner --loop --interval-minutes 10
--lookback-hours 1
```

Env: OFDD_FDD_INTERVAL_MINUTES, OFDD_FDD_LOOKBACK_HOURS.

Each cycle:

1. RuleStore().list_rules() — enabled rules only
2. For each rule × resolved site: load feather frame → prepare_fdd_rows → playground.sweep_rule() (or script mode)
3. Code from read_source(source_path) — same .py the browser saved
4. save_results() → fdd_results.json → / check-engine light

Ansible edge: openfdd-fdd-loop.timer runs --once on an interval.

96.4 AI agent — same files

Two surfaces; both hit the bridge, not Codex in the browser.

96.4.1 1. AI Agent chat tab (/agent)

- POST /openfdd-agent/chat — Ollama only; **does not auto-call tools**.
- GET /openfdd-agent/context — lists saved_rules, fault_codes, available tools for the system prompt.
- To **write** rules from automation, call tools explicitly (see below) or use Rule Lab APIs with integrator/agent auth.

96.4.2 2. Agent tools (agent role)

POST /openfdd-agent/tool with JSON body { "tool": "...", "args": { ... } }:

Tool	Effect
rules.save	RuleStore().upsert() — same path as Rule Lab save ; writes JSON + rules_py/*.py
rules.run_batch	Runs fdd_runner.run_batch() → updates check-engine
model.add_*	BRICK model CRUD
app.edit_file	Edit files under workspace/ only if OFDD_AGENT_ALLOW_APP_EDIT=1

Read source without a dedicated tool:

- GET /api/rules/saved/{id}/source (integrator/agent auth)
- Or read workspace/data/rules_py/*.py on the host (Codex shell, SSH, IDE)

Human and AI see identical .py files because both persist through RuleStore.upsert() → rule_source.write_source().

96.4.3 Codex agent shell (repo checkout)

openfdd-agent-shell follows AGENTS.md → [rules-crud-and-batch-run skill](#). Scratch experiments go in workspace/scratch/; promote keepers via POST /api/rules/save or rules.save tool.

96.5 Data pipeline (BACnet → rules)

BACnet poll (commission agent, 60s)

→ workspace/bacnet/polls/samples.csv

Bridge poll worker (on CSV mtime)

→ feather_store/bacnet/<site_id>/latest.feather

FDD loop / Rule Lab test / batch

→ reads feather + rules_py/*.py

→ fdd_results.json → check-engine

Enable points in workspace/bacnet/commissioning/points.csv; poll
status: GET <http://127.0.0.1:8767/api/bacnet/poll/status>.

96.6 API quick reference

GET /api/rules/saved

GET /api/rules/saved/{id}/source

PUT /api/rules/saved/{id}/source

POST /api/rules/save

DELETE /api/rules/saved/{id}

POST /api/rules/bindings

POST /api/rules/batch

POST /api/playground/lint

POST /api/playground/test-rule

POST /api/playground/run-script

GET /openfdd-agent/context

POST /openfdd-agent/tool # agent role

POST /openfdd-agent/chat

96.7 See also

- [Operator dashboard \(Rule Lab\)](#) — stack start, tabs, auth
- [Building check-engine light](#) — fault_code tagging
- [Verification](#) — lint/save/batch smoke checks
- [Agent & operator playbook](#) — MCP + bridge discovery
- Skill: [rules-crud-and-batch-run](#)

97 Ollama on the edge (hardware)

98 Ollama on the edge (hardware)

Open-FDD uses the official `ollama/ollama` image or a **host binary** from Ansible `ollama_bootstrap.yml`. Purpose: Local Ollama (check-engine) — not deployment AI.

98.1 Recommended by hardware

Situation	Approach
x86/ARM VM with NVIDIA GPU	Host systemd Ollama (<code>openfdd_docker_ollama: false</code>) + <code>deploy.sh ai</code>
No GPU / small Pi	Docker ollama service or disable AI
GPU in Docker	NVIDIA Container Toolkit + <code>ollama_gpu_mode: gpu</code>

98.2 Variables (host_vars / group_vars)

Variable	Role
<code>enable_ollama</code>	Turn on AI stack
<code>openfdd_docker_ollama</code>	Host vs compose Ollama
<code>ollama_gpu_mode</code>	<code>cpu</code> <code>auto</code> <code>gpu</code>
<code>ollama_ram_tier</code>	Default model tier

98.3 Acme example

```
./scripts/docker_build.sh --save
cd infra/ansible && source secrets/acme.env.local
./deploy.sh docker --limit acme_vm_bbartling
./deploy.sh ai --limit acme_vm_bbartling
```


98.4 Maintenance

- ollama pull <model> on host or docker compose exec ollama ollama pull ...
- Logs: journalctl -u ollama -f or docker compose logs -f ollama

99 Edge Deploy

100 Edge deploy (Ansible)

Public reference for deploying Open-FDD to field VMs and bench hardware. **Real IPs, SSH passwords, and BACnet OT addresses live only in gitignored files** on your control machine — never in this doc or on GitHub.

100.1 What is safe on GitHub

Tracked	Gitignored
infra/ansible/deploy.sh, playbooks, *.example inventory/host_vars	inventory.yml, host_vars/ acme_vm_bbartling.yml
infra/ansible/secrets/ *.example, secrets/ README.md	infra/ansible/secrets/*.env.local
inventory.example.yml (placeholder IPs)	edge_backup/local/**
docs/edge_deploy.md (this file)	workspace/auth.env.local

100.2 One-time setup

```
cd infra/ansible

cp inventory.example.yml inventory.yml
cp host_vars/acme_vm_bbartling.yml.example host_vars/
acme_vm_bbartling.yml
cp secrets/acme.env.example secrets/acme.env.local

# Edit inventory.yml: set ansible_host + ansible_user for each
host

# Edit secrets/acme.env.local: SSHPASS, ACME_SSH_HOST, BACnet
```

bind, dashboard URL
chmod 600 secrets/*.env.local

100.3 Deploy commands (examples)

Build UI first when deploying ui or all:

```
../../scripts/build_operator_dashboard.sh prod
./deploy.sh ui --limit acme_vm_bbartling
./deploy.sh backend --limit acme_vm_bbartling
./deploy.sh commission --limit acme_vm_bbartling
./deploy.sh drivers --limit acme_vm_bbartling -e
enable_bacnet_poll_driver=true
```

deploy.sh sources secrets/acme.env.local when --limit
acme_vm_bbartling is set, so you do not need to re-export SSHPASS
every session if the file exists.

100.4 Inventory hosts

Host key	Role	Example placeholder in repo
acme_vm_bbartling	Real building (x86 VM, OT BACnet)	198.51.100.55 in inventory.example.yml
bacnet_pi	Boss Pi bench (armv7l)	198.51.100.12

Set real addresses in private inventory.yml. Use [RFC 5737 documentation addresses](#) in examples only.

100.5 Acme commissioning layout

Commissioning CSVs are pushed from:

```
edge_backup/local/<site_id>/<building_id>/points.csv
edge_backup/local/<site_id>/<building_id>/device_poll_profiles.csv
```

Example site/building ids: acme / vm-bbartling (set in host_vars, not secrets).

100.6 Auth vs SSH

- **SSH** (SSHPASS, acme.env.local): VM login for Ansible — field IT credential.
- **Dashboard** (workspace/auth.env.local): integrator/operator/agent web login — deployed via config tag, separate from SSH.

Do not put web passwords in secrets/*.env.local or SSH passwords in auth.env.local.

100.7 Agent / Cursor context

- Read infra/ansible/secrets/README.md and the host's secrets/<site>.env.local before deploy.
- Read AGENTS.md § Security.
- Rule: never commit or quote real secrets in tracked files.

Full playbook reference: [infra/ansible/README.md](#).

101 Security Hardening

102 Operator Bridge security hardening

The Open-FDD bridge and React dashboard run on BACnet/OT edge hosts. Default posture is **fail closed**: authentication required, BACnet writes off, Rule Lab execution bounded, minimal error detail to browsers.

102.1 Required for LAN / edge production

Variable	Purpose
OFDD_AUTH_SECRET	HMAC signing key (32+ characters)
OFDD_OPERATOR_USER / OFDD_OPERATOR_PASSWORD	Read-mostly operator role
OFDD_INTEGRATOR_USER / OFDD_INTEGRATOR_PASSWORD	Rule Lab, model import, BACnet commission

Variable	Purpose
OFDD_AGENT_USER / OFDD_AGENT_PASSWORD	Agent tools and app-edit gates

Set in workspace/auth.env.local (gitignored). Caddy should reverse-proxy to the bridge; do not expose port 8765 to the internet without TLS and auth.

102.2 Dev-only (localhost)

Variable	Purpose
OFDD_AUTH_DISABLED=1	Skip Bearer tokens only when OFDD_BRIDGE_HOST is 127.0.0.1 / localhost
OFDD_INSECURE_LAN_DEV=1	Allows OFDD_AUTH_DISABLED on non-localhost bind (lab only)
OFDD_BRIDGE_HOST=127.0.0.1	Recommended for local uvicorn / compose on one machine

Missing auth on 0.0.0.0 bind logs a warning; set
OFDD_AUTH_STRICT_STARTUP=1 to **fail startup** instead.

102.3 CORS

Variable	Purpose
OFDD_CORS_ORIGINS	Comma-separated browser origins (e.g. http:// 192.168.1.10:5173)
OFDD_CORS_ALLOW_PRIVATE_LAN=1	Opt-in: add detected LAN IP origins (not implied by 0.0.0.0 bind)

102.4 BACnet writes

Variable	Purpose
OFDD_ENABLE_BACNET_WRITE=1	Enable POST /api/bacnet/write (default off)

Optional allowlist: workspace/bacnet/write_allowlist.json with
device_instances and/or object_identifiers.

102.5 Rule Lab / diagnostics

Variable	Purpose
OFDD_PLAYGROUND_TIMEOUT_S	Total rule/script budget (default 30s)
OFDD_PLAYGROUND_MEMORY_MB	Child process RSS cap (default 512)
OFDD_PLAYGROUND_SUBPROCESS=0	Disable OS subprocess isolation (not recommended on edge)
OFDD_PLAYGROUND_INPROCESS=1	Run in API process (localhost dev/tests only)
OFDD_DEBUG_TRACEBACKS=1	Return tracebacks to browser (dev only)
OFDD_DEBUG_DIAGNOSTICS=1	Expose repo_root in agent context (integrator/agent)

102.6 Agent app edit

Variable	Purpose
OFDD_AGENT_ALLOW_APP_EDIT=1	Allow agent tools that modify app source (default off)

102.7 Ollama / AI

Use workspace/ollama.env.local. Home dashboard uses read-only **building insight** (no chat). Interactive LLM chat is on the **AI Agent** tab only.